

分类号 _____

学校代码 10487

学号 M201676105

密级 _____

华中科技大学

硕士学位论文

基于 Spring Boot 的校园轻博客系统 的设计与实现

学位申请人：邓 笑

学 科 专 业：软件工程

指 导 教 师：卢 力 副教授

答 辩 日 期：2018.12.19

**A Thesis Submitted in Partial Fulfillment of the Requirements for
the Degree for the Master of Engineering**

**Design and Implementation of a Campus Light
Blog System Based on Spring Boot**

Candidate : Deng Xiao

Major : Software Engineering

Supervisor : Assoc. Prof. Lu Li

Huazhong University of Science & Technology

Wuhan 430074, P.R.China

December, 2018

摘要

随着互联网的普及，网络几乎成为了人们搜集信息和交流沟通最方便最快捷的通道，网络同时也给了人们充分表达自我的舞台。在这样的全民交流的时代背景下，在校园里开辟一块属于全校师生的交流天地，为老师之间、学生之间、以及师生之间的交流建立一个简单干净的网络平台，是具有重要意义的。基于此目的，同时轻博客具有多样表达以及简单社交等特点，Spring Boot又是轻量级框架，具有方便开发和维护等特点，所以设计并实现了基于Spring Boot的校园轻博客系统。

根据软件生存周期的流程，首先对系统进行需求分析，分别对系统的功能、性能以及安全需求进行分析，然后对系统进行总体设计，将整个系统分为系统管理模块、博文管理模块、标签管理模块、互动管理模块以及用户信息管理模块，并对具体的模块进行详细设计，接着借助开发工具实现系统的博文浏览、博文搜索、博文编辑，以及用户管理和博客管理等核心功能，最后借助软件测试方法对系统的功能、性能以及安全性进行测试。

总的来说，校园轻博客系统的功能基本实现，同时性能和安全性也得到了相应的保障：系统借助 Spring Boot+MyBatis 开发框架使得代码结构低耦合，系统易于开发和维护；前端框架以及多种插件使得系统支持多种发布形式，界面简洁、操作简单；Apache Shiro 安全框架和过滤器等多种技术保障系统安全。

关键词：轻博客 Spring Boot 框架 软件测试 安全框架

Abstract

With the popularity of the Internet, the Internet has become the most convenient and fastest channel for people to collect information and communicate. The network also gives people a stage to fully express themselves. In the context of such a nationwide exchange, it is important to open up a network of exchanges between teachers and students throughout the school, and to establish a simple and clean network platform for communication between teachers, students, and between teachers and students. meaningful. For this purpose, while the light blog has a variety of expressions and simple social features, Spring Boot is a lightweight framework, with convenient development and maintenance features, so designed and implemented a Spring Boot-based campus light blog system.

According to the software life cycle process, the system first analyzes the requirements of the system, analyzes the functions, performance and security requirements of the system, then designs the system, and divides the whole system into system management module, blog management module and label management module. , interactive management module and user information management module, and detailed design of specific modules, and then use the development tools to achieve the system's blog browsing, blog post search, blog post editing, as well as user management and blog management and other core functions, and finally by means of software testing methods Test the functionality, performance, and security of your system.

In general, the functions of the campus light blog system are basically realized, and the performance and security are also guaranteed accordingly: the system uses the Spring Boot+MyBatis development framework to make the code structure low-coupling, the system is easy to develop and maintain; the front-end framework and many The plug-in enables the system to support multiple publishing formats, with a simple interface and simple operation; Apache Shiro security framework and filters and other technologies to ensure system security.

Key words: Light blog Spring Boot framework Software test
Security framework

目 录

摘 要.....	I
Abstract.....	II
1 绪论	
1.1 研究背景与意义.....	(1)
1.2 国内外研究概况.....	(2)
1.3 主要研究内容.....	(3)
2 关键技术分析	
2.1 Web MVC 模式.....	(5)
2.2 Spring Boot 框架.....	(7)
2.3 MyBatis 框架.....	(9)
2.4 Apache Shiro 框架.....	(11)
2.5 本章小结.....	(12)
3 系统分析与设计	
3.1 系统需求分析.....	(13)
3.2 系统总体设计.....	(19)
3.3 系统功能模块设计.....	(21)
3.4 系统数据库设计.....	(31)
3.5 系统安全设计.....	(37)
3.6 本章小结.....	(39)
4 系统实现与测试	
4.1 系统开发环境.....	(40)
4.2 系统功能模块实现.....	(40)

华中科技大学硕士学位论文

4.3 系统安全实现.....	(54)
4.4 系统测试.....	(58)
4.5 本章小结.....	(63)
5 总结与展望	
5.1 全文总结.....	(64)
5.2 展望.....	(65)
致 谢	(66)
参考文献	(67)

1 绪论

1.1 研究背景与意义

高校中的学生正处于寻找自我的阶段，刚成年或已成年，思想或许正处于萌芽期，急于向世界宣告自己独立而特别的存在；或许少年老成，已学会表达自我多时。同时，如今的网络时代也已向人们提供了大量的方式去认识世界、感知世界，如微博，刷一刷即可了解此刻最新最劲爆的消息；微信，摇一摇即可结识身边或近或远的人们；抖音，拍一拍即可记录或感动或搞笑的瞬间。轻博客作为一种介于微博和博客之间的一种网络社交模式，它既继承了博客的表达基因，注重内容为主以及兴趣驱动，同时保留了微博的传播基因，保留用户之间的互动，包括评论、喜欢、转发等特点，使其具有线上“俱乐部”之称，以兴趣主导，以精致表达吸引围观，进而形成互相分享、互相学习、互相探讨的良好社交氛围。

高校的学生们所能获取信息以及表达自我的网络媒体通道有贴吧、BBS、微博公众号、微信公众号、人人网、QQ群及其公众号等方式，但无一种是师生们能全体参与并且自由表达的方式。贴吧多为学生“地下组织”，虽也有管理员管理，但信息鱼龙混杂，有效信息很难获取；BBS虽含有大量信息，但因其编辑复杂以及其界面较为陈旧，很难吸引学生长久驻留；各类公众号多用于发布通知，且具有一定的工作性质，这使得公众号维护者必须定期更新，同时务必保证信息准确。

校园轻博客系统的意义主要体现在以下几个方面：

（1）轻博客帮助学生养成文字表达的行为习惯。轻博客作为比博客相对轻松，比微博相对严肃的方式，用精致内容表达着对世界的认知，表达着对自我的探寻，表达着对梦想的追寻，表达着对现实的抵抗。文字图片的排版以及内容编排需要静下来去思考、去沉淀，这其实给了高校的学生们一种去浮躁的表达方式，也是帮助学生养成文字表达的行为习惯。

（2）校园环境使得交流更加安全可靠。通过表达认识一批志同道合的伙伴们，这些小伙伴可能是比你小一两届的学弟学妹们，可能是比你高几届的学长学姐们，

也可能是已“驻守”学校多年的老师们，线上大家通过文字交流，线下可以约三五老友相聚，因为是校园环境的关系，这样的群体更容易聚集，也更加放心交流。

(3) 校园轻博客倡导积极表达，净化校园表达环境。校园轻博客系统鼓励大家传达正能量、分享美好生活、分享兴趣爱好，同时轻博客鼓励大家专注于兴趣分享，通过兴趣标签找到校园里的知己们。

1.2 国内外研究概况

博客，又称部落格，是从Blogger音译而来，Jorn Barger在1997年首次提出weblog，Evan Williams在2006年创立了Twitter，David Karp在2007年创建了介于Twitter和blogger之间的Tumblr^[1-3]。在国内，2000年weblog的概念首次进入中国，2002年博客因博客中国的开通从而进入全面推广时期；2005年各大门户网站纷纷构建自己的博客系统，进入博客时代^[4,5]。2007年开始出现像饭否等独立微博的踪影，2009年微博因新浪微博的上线开始在全国流行^[6]。2011年出现第一个轻博客——点点网，同一个时期各大网站纷纷推出了自己的轻博客，如网易的乐乎，新浪Qing等，轻博客开始受到关注^[7,8]。

博客在教育应用中占据其独特优势。首先，博客打破了话语权的绝对权威性，在传统的教学模式下，老师具有绝对的权威性，学生的表达多半会被忽略，甚至可能出现嘲笑，严重时可能出现被孤立等情况产生，而博客因其表达性和匿名性，可以有效的进行讨论相关争议，并能吸引“支持者”，同时引起教育者的关注，并获得平等的对话权；其次，博客能够促进Learner to Learner的学习模式，每个人都可以是老师，每个人都可以参与其中，共同学习，共同进步；再次，博客给予了学生尝试网络学习的指导性，帮助学生在纷繁复杂的网络世界中找到有效信息；最后博客促进了学生的成长，记录下自己的所思所感，并借助记录可以进行一个不断反思自省的阶段，进而获得成长^[9,10]。

博客对校园文化也产生了一定积极影响。校园文化，作为一种特殊的社会文化，它将校园作为活动范围，校内人员作为活动主体，围绕校园精神开展校园活动。校园文化作为学校的重要组成部分，由学校师生共同创造，并且体现着学校对外展现

的精神面貌。网络对高校师生们的学习生活产生了深刻的影响，而博客作为一个可以记录和分享知识、生活的平台，为高校的老师们提供了一种新的教学方式和平台，为不同的思想交流提供了广阔的空间，为深化学生们的专业认识和拓宽知识面提供了可能^[11]。

博客在校园中的发展已有一段时间，足以说明博客与校园的结合是可以被接受，同时具有其独特作用，如教师博客可以弥补课堂教育的局限性，班级博客给家长们提供了一个了解孩子在校状况的通道，而学科博客可以提升学科教学研究的学术性和趣味性，实现学科科研的“总动员”^[12-14]。

但这样的博客大多数是以老师或专业人士为主，以学生为辅，师生的互动性以及学生参与其中的主动性相对而言并没有达到预期的要求。轻博客作为博客和微博的延申，虽同样强调内容，但更多以兴趣驱动，鼓励用户主动的去了解，去学习，去交流所喜欢的、所感兴趣的内容，只有当学生主动参与其中，其互动性才会得到实质性的进展。同时，作为新一代的社交模式，轻博客在很多方面都表现出了自己的独特性：

（1）重图片轻文字，阅读体验更轻松：图片表达所带来的视觉冲击性以及生动形象性，使得浏览图片比阅读文字更适合当代人文化消费的快节奏，阅读体验更轻松；

（2）重内容轻关系，致力于高质量内容的发布与传播：以兴趣为驱动的社交，相对更强调内容而非人本身的设计模式使得内容的发布更个性化，内心驱动使得内容更高质量化、更自主创作化；

（3）信息需求趋同，形成深度交流和共建关系：通过轻博客发现志趣相投的人们互动会更加频繁，关系相对更加紧密^[15]。

轻博客与校园的结合弥补了校园博客环节中所缺失的学生自主表达的环节，并且在某种程度上拉近了高校中学生与学生之间的距离。

1.3 主要研究内容

本课题将应用软件工程中的面向对象的方法，对基于 Spring Boot 的校园轻博客

系统进行分析与设计，借助 IDEA 工具与 Java 高级语言进行代码编写，使用当下较为先进的 Spring Boot + MyBatis 框架进行开发实现，具体的研究内容如下：

（1）对校园轻博客系统中需要使用的关键技术进行学习研究。Spring Boot + Mybatis 框架的本质是 MVC 框架，对整个代码结构搭建松耦合的三层，Controller、View、Model 层，完成 View 与 Controller 以及 Controller 与 Model 交互结构的搭建，进而完成整个系统的基本框架搭建，再根据具体需求对框架进行添加相关结构等操作，将采用 MAVEN 管理整个项目的生命周期；

（2）对校园轻博客系统进行需求分析，对系统所具备的核心功能进行分析总结，并对系统的性能以及系统应具备的安全性进行分析总结，同时借助 Visio 工具绘制相关图表进行辅助说明；

（3）对校园轻博客系统进行总体设计：对系统的架构和功能结构进行设计，并针对每个功能模块进行详细设计；对系统进行数据库设计：设计实体结构，设计实体与实体之间的关系，同时借助 PowerDesigner 绘制出对应的逻辑模型和物理模型；对系统进行安全性设计：设计分层结构、多重验证、安全编码等保障系统安全；

（4）对校园轻博客系统进行系统实现，借助开发工具以及之前的系统设计说明实现系统的核心功能以及安全性，并且通过对各个模块关键功能的业务逻辑实现描述以及借助绘图工具绘制的相关图表来说明整个系统功能实现以及安全实现的过程；

（5）对校园轻博客系统进行系统测试，通过黑盒测试和白盒测试对系统分别进行功能和代码结构测试，并通过功能测试用例、性能测试用例以及安全测试用例对测试过程进行具体的说明，最后根据测试结论修复系统问题。

2 关键技术分析

本章将阐述校园轻博客系统实现所使用的关键技术，包括但不限于 Java Web 开发所使用的经典模式 Web MVC，以及用于基本框架搭建的 Spring Boot 和 MyBatis 框架，还有为加强系统安全使用的 Apache Shiro 框架。

2.1 Web MVC 模式

MVC 的完整描述为 Model View Controller。MVC，作为一种常用于开发图形用户界面（GUI）的体系架构模式，将应用程序划分为三个相互连接的部分，使得信息的内部表示与向用户呈现和接受信息的方式分开^[16]。这种架构现在频繁用于设计 Web 应用程序、PC 端、移动端等各类客户端，甚至根据具体需求在此基础上扩展出了不同的架构，如 MVP 等。

MVC 包含了模型（model）、视图（view）以及控制器（controller）三个部分。模型，作为 MVC 结构的核心组成部分，管理着系统的数据以及业务逻辑；视图是一种数据的呈现方式，例如视图、图表等；控制器控制着模型与视图之间的转换，接收输入并将其转换为模型或视图的命令。这三者的交互模式为模型从控制器接收用户输入，控制器响应用户输入并对数据模型对象执行交互，然后将输出传递给模型，视图以特定的格式呈现模型^[17]。

Web MVC 是将 MVC 应用于 Web 环境，当 Web Objects 移植到 Java 时，MVC 模式开始受到 Java 开发人员的欢迎。后来的 Java 框架，如 Spring（2002 年 10 月发布），延续了 Java 和 MVC 之间的紧密联系。Django（2005 年 7 月，Python 开发）和 Rails（2005 年 12 月，Ruby 开发）框架的引入，两者都强调快速部署，增加了 MVC 在长期以来流行的传统企业环境之外的受欢迎程度。

Web MVC 的基本工作流程如图 2.1 所示：

- （1）Controller 截获用户发出的请求；
- （2）Controller 调用 Model 完成状态的读写操作；
- （3）Controller 把数据传递给 View；
- （4）View 渲染最终结果并呈献给用户。

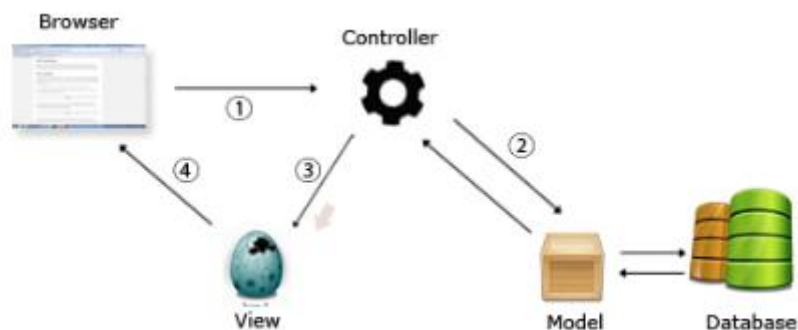


图 2.1 Web MVC 结构图

MVC 模式常常伴随着三层架构。三层架构是分层结构中很常用的一种结构，其结构如图 2.2 所示，是在用户访问端与数据访问端之间插入一个中间层，从而分为表示层、业务逻辑层以及数据访问层^[18]。

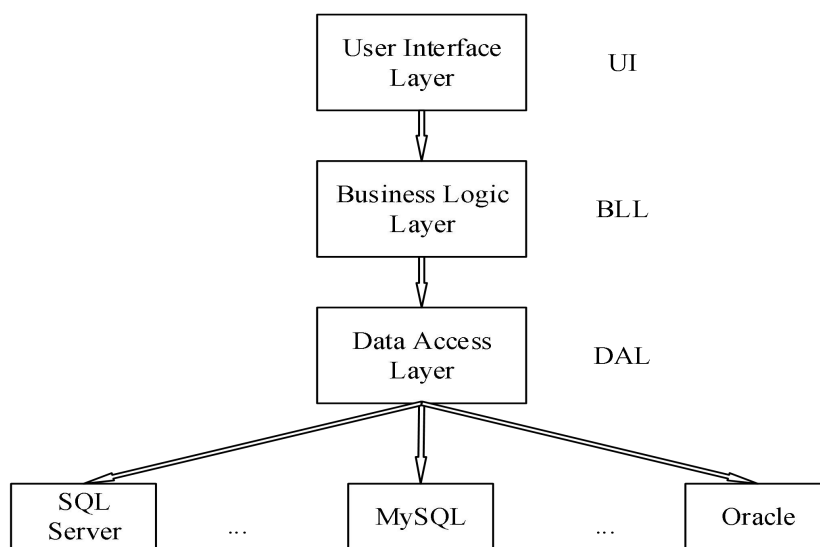


图 2.2 三层架构图

表示层（UI），展示给用户的界面；业务逻辑层（BLL），具体的业务逻辑实现操作；数据访问层（DAL），访问数据库细节的封装，为表示层和逻辑层提供数据服务。

事实上，MVC 的三层与三层架构并没有直接相关关系，但因为其本质均是对系统结构进行分层和解耦操作，所以我们常常将 MVC 与三层架构联系在一起。三层架构与 MVC 的层次映射关系如图 2.3 所示，MVC 的 View 和 Controller 都对应着三层

架构中的 UI, View 管理着界面的显示, 而 Controller 控制着客户端数据到服务器端数据的交互, 相当于 UI 中的 Servlet; MVC 的 Model 对应着三层架构中的 BLL 和 DAL, 将数据分为两类, 一类用于处理业务逻辑, 一类用于处理数据库访问。

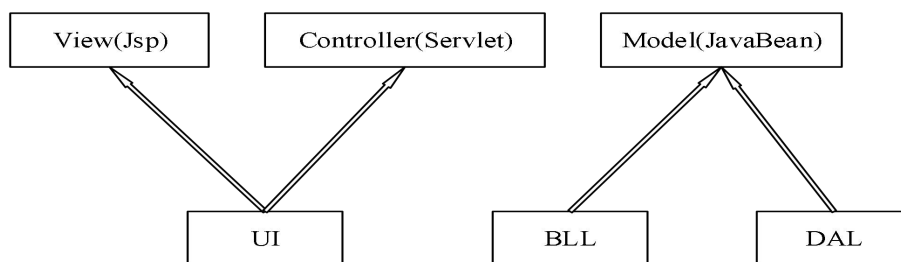


图 2.3 三层架构与 MVC 的层次映射关系图

MVC 模式被广泛用于 Java Web 的各种框架中, 如 SSH (Struts+Spring MVC+Hibernate)、SSM (Spring+SpringMVC+Mybatis) 等等。Java Web 开发框架是将 MVC 模式的思想抽象出来, 根据代码实际功能分化成不同的部分, 由不同的组件承担对应的部分, 并将这些组件高效地集成于一体^[19,20]。

2.2 Spring Boot 框架

2.2.1 Spring 框架

Spring 框架是 Java 平台上的一种开源应用框架, 提供具有控制反转特性的容器^[21]。尽管 Spring 框架自身对编程模型没有限制, 但其在 Java 应用中的频繁使用让它备受青睐, 以至于后来让它作为 EJB (Enterprise JavaBeans) 模型的补充, 甚至是替补。

Spring 框架为开发提供了一系列的解决方案, 比如利用控制反转的核心特性, 并通过依赖注入实现控制反转来实现管理对象生命周期容器化, 利用面向切面编程进行声明式的事务管理, 整合多种持久化技术管理数据访问, 提供大量优秀的 Web 框架方便开发等等^[22,23]。

Spring 框架具有控制反转 (IOC) 特性, IOC 旨在方便项目维护和测试, 它提供了一种通过 Java 的反射机制对 Java 对象进行统一的配置和管理的方法。Spring 框架

利用容器管理对象的生命周期，容器可以通过扫描 XML 文件或类上特定 Java 注解来配置对象，开发者可以通过依赖查找或依赖注入来获得对象。

Spring 框架具有面向切面编程（AOP）框架，Spring AOP 框架基于代理模式，同时运行时可配置；AOP 框架主要针对模块之间的交叉关注点进行模块化。Spring 框架的 AOP 框架仅提供基本的 AOP 特性，虽无法与 AspectJ 框架相比，但通过与 AspectJ 的集成，也可以满足基本需求。Spring 框架下的事务管理、远程访问等功能均可以通过使用 Spring AOP 技术实现。

Spring 的事务管理框架为 Java 平台带来了一种抽象机制，使本地和全局事务以及嵌套事务能够与保存点一起工作，并且几乎可以在 Java 平台的任何环境中工作。Spring 集成多种事务模板，系统可以通过事务模板、XML 或 Java 注解进行事务配置，并且事务框架集成了消息传递和缓存等功能。

Spring 的数据访问框架解决了开发人员在应用程序中使用数据库时遇到的常见困难。它不仅对 Java:JDBC、iBATS/MyBATIs、Hibernate、Java 数据对象（JDO）、Apache OJB 和 Apache Cayne 等所有流行的数据访问框架中提供支持，同时还可以与 Spring 的事务管理一起使用，为数据访问提供了灵活的抽象。

Spring 框架最初是没有打算构建一个自己的 Web MVC 框架，其开发人员在开发过程中认为现有的 Struts Web 框架的呈现层和请求处理层之间以及请求处理层和模型之间的分离不够，于是创建了 Spring MVC^[24]。

2.2.2 Spring Boot 框架

Spring Boot 是由 Pivotal 团队在 2013 年开始研发、2014 年 4 月发布第一个版本的全新开源的轻量级框架。它基于 Spring 4.0 设计，不仅继承了 Spring 框架原有的优秀特性，而且还通过简化配置来进一步简化了 Spring 应用的整个搭建和开发过程。另外 Spring Boot 通过集成大量的框架使得依赖包的版本冲突，以及引用的不稳定性等问题得到了很好的解决。

Spring Boot 所具备的特征有：

（1）可以创建独立的 Spring 应用程序，并且基于其 Maven 或 Grandel 插件，可以创建可执行的 JARs 和 WARs；

- (2) 内嵌 Tomcat 或 Jetty 等 Servlet 容器;
- (3) 提供自动配置的“starter”项目对象模型 (POMS) 以简化 Maven 配置;
- (4) 尽可能自动配置 Spring 容器;
- (5) 提供准备好的特性, 如指标、健康检查和外部化配置;
- (6) 绝对没有代码生成, 不需要 XML 配置^[25]。

Spring Boot 框架中还有两个非常重要的策略: 开箱即用和约定优于配置。开箱即用, Out of box, 是指在开发过程中, 通过在 MAVEN 项目的 pom 文件中添加相关依赖包, 然后使用对应注解来代替繁琐的 XML 配置文件以管理对象的生命周期。这个特点使得开发人员摆脱了复杂的配置工作以及依赖的管理工作, 更加专注于业务逻辑。约定优于配置, Convention over configuration, 是一种由 Spring Boot 本身来配置目标结构, 由开发者在结构中添加信息的软件设计范式。这一特点虽降低了部分灵活性, 增加了 BUG 定位的复杂性, 但减少了开发人员需要做出决定的数量, 同时减少了大量的 XML 配置, 并且可以将代码编译、测试和打包等工作自动化^[26]。

Spring Boot 应用系统开发模板的基本架构设计从前端到后台进行说明: 前端常使用模板引擎, 主要有 FreeMarker 和 Thymeleaf, 它们都是用 Java 语言编写的, 渲染模板并输出相应文本, 使得界面的设计与应用的逻辑分离, 同时前端开发还会使用到 Bootstrap、AngularJS、JQuery 等; 在浏览器的数据传输格式上采用 Json, 非 xml, 同时提供 RESTful API; Spring MVC 框架用于数据到达服务器后处理请求; 到数据访问层主要有 Hibernate、MyBatis、JPA 等持久层框架; 数据库常用 MySQL; 开发工具推荐 IntelliJ IDEA^[27]。

2.3 MyBatis 框架

MyBatis 是一种 Java 持久层框架, 借助 XML 描述符或特定注解将对象与 SQL 语句或存储过程耦合在一起。MyBatis 是 iBATIS 3.0 的一个分支, 由一个包含 iBATIS 原始创建者的团队维护。

MyBatis 所具备的特点有:

- (1) 传统的 ORM 框架是将 Java 对象结构映射为数据库的表结构, 然后调用封装好的接口调用数据库操作; 而 MyBatis 是将 Java 接口映射为对应操作的 SQL 语句, 然后在数据库中执行 SQL 语句;

(2) MyBatis 允许使用所有的数据库功能，如视图、复杂查询、存储过程等；MyBatis 在一些遗留系统、运用了非规范化的数据库以及需要对 SQL 有完全控制权的系统上是可以很好的发挥自身特性；

(3) MyBatis 在编码上比 JDBC 简单，执行 SQL 语句只需要一行代码；MyBatis 通过映射引擎并以声明的方式对 Java 对象和 SQL 执行结果进行映射；MyBatis 支持动态构建 SQL 语句；

(4) MyBatis 与 Spring Framework 和 Google Guice 集成，此功能允许用户构建没有依赖关系的业务代码；

(5) MyBatis 支持声明性数据缓存。可通过对语句进行标记，以便数据库检索可直接从高速缓存中获取，而非数据库；

(6) MyBatis 包含了一个基于 Java HashMap 实现的默认缓存，并且提供了一个 API 来集成其他缓存实现，以及用来集成的 OSCache、Memcached 等默认连接器^[28]。

MyBatis 由基础支撑层、数据处理层以及接口层三层架构组成，其功能架构图如图 2.4 所示。

基础支撑层，是数据处理层的基础支撑，主要包含加载配置文件、管理数据库的连接、管理事务和管理缓存等操作；数据处理层主要包含查找、解析和执行 SQL 语句，以及根据 SQL 执行结果映射 Java 对象等操作；接口层主要向外提供可用 API，开发者可利用本地 API 更加方便快捷地操纵数据库^[29,30]。

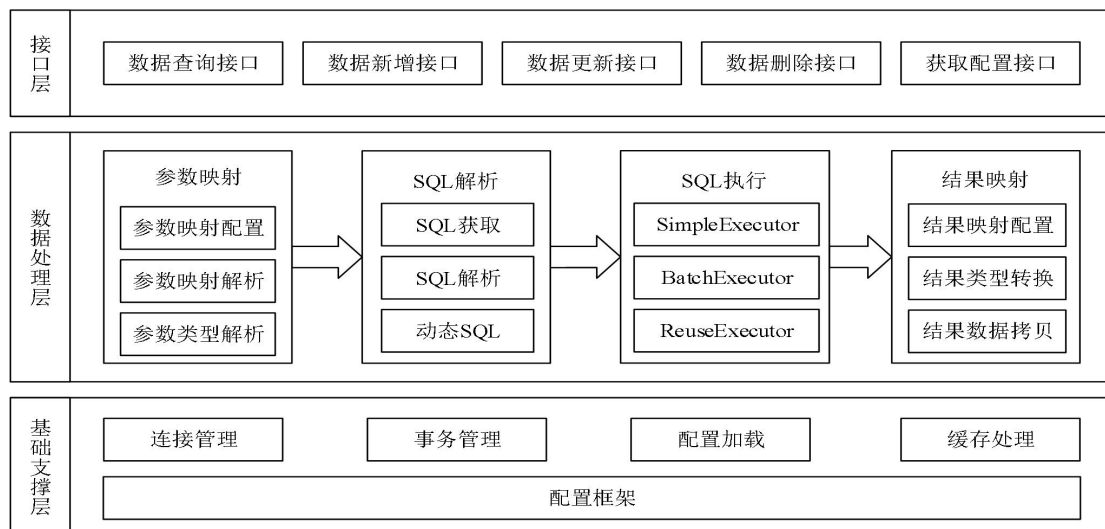


图 2.4 MyBatis 功能架构图

MyBatis 的基本工作流程为：首先加载相关 XML 配置文件并初始化，生成相应 MappedStatement 对象；其次当用户请求调用 MyBatis 提供的 API，根据 SQL 的 ID 找到对应的 MappedStatement 对象，并将相关参数传给数据处理层处理；再次处理操作请求：建立连接→参数映射→SQL 解析→SQL 执行→结果映射→释放连接；最后返回处理结果。

2.4 Apache Shiro 框架

Apache Shiro 是一个开源的 Java 安全框架，具有身份验证、授权、加密和会话管理等功能。Shiro 不仅直观易用，同时提供健壮的安全特性，并且可以保护任何应用程序——从命令行应用程序，移动应用程序到最大的 Web 和企业应用程序。

Apache Shiro 的架构有三个主要概念：Subject、SecurityManager 和 Realms。Subject 是一个安全术语，大致上是指“当前正在执行的用户”。用户的概念大多与人类相关，但在安全领域里，Subject 可以指人类，也可以指第三方流程、守护进程账户、或者类似的东西，所以 Subject 指的是当前与软件交互的东西。SecurityManager 是 Subject 背后对应实际操作部分，SecurityManager 管理着所有的安全操作，无论是当前用户的还是其他用户的；SecurityManager，作为 Shiro 的核心部分，管理着所有 Subject 和 Shiro 组件，并保证它们按照安全规则进行交互。Realms 连接着 Shiro 和当前系统的安全数据，即当用户执行相应安全操作时，Shiro 会从配置的一个或多个 Realms 中查找预存安全数据^[31]。

Apache Shiro 提供了与应用程序安全相关的 API 来实现 Shiro 四项重要功能，以及部分辅助特性。Shiro 拥有认证（Authentication）、授权（Authorization）、加密（Cryptography）和会话管理（Session Management）功能，以及 Web 应用程序安全，单元测试和多线程支持等辅助特性。

认证，是验证用户身份的过程，即登录过程，通常分为三步：第一步，收集主体，即用户的标识信息，以及凭证，即支持身份的证明；第二步，将主体和凭证交给系统；第三步，根据主体和凭证与预期对应数据的匹配情况做出相应决策。当系统在对登录用户进行身份验证时，通常可以利用“用户名+密码”的组合来进行身份验证，

但同时 Shiro 也允许用户根据具体业务需求制定相应的验证方法^[32]。

授权，本质上是访问控制，控制用户在应用程序上可以访问的内容，例如资源、网页等。这一功能可以通过为用户定义角色以及赋予权限的操作以控制用户的行为，同时也表明开发人员可以借助 Subject API 对角色和权限进行检查。Shiro 可以利用支持简单通配符的权限表达式来进行权限描述，以达到灵活配置的目的^[33]。

加密，是隐藏或混淆数据的过程，以防非法窥探。Shiro 希望在加密方面做到尽可能地简化，并使 JDK 的加密支持成为可能。Shiro 引入了 CipherService API，让用户仅提供密钥即可进行加密和解密操作。Shiro 密切关注其加密支持的领域是 Hash 加密（又称消息摘要）和 cipher 加密，还封装了一些像 MD5 等加密算法，以便用户使用^[34]。

会话管理，Shiro 在安全框架领域提供了独特功能：在任何应用程序和任何架构层中将使用一致的 Session API，这也就意味着开发人员可在任意层使用统一的 Session API，而不用 Servlet 或 EJB 特定的机制，这也同时体现了 Shiro Session 最重要的特点之一：容器无关性。这使得应用部署过程更加简单方便^[35]。

Shiro 可以仅通过在 web.xml 中定义 Servlet 的 IniShiroFilter 即可集成于 web 环境中；Shiro 支持多线程环境下的并发验证；Shiro 同样支持各类测试操作^[36]。这几大特性和其他相关特性强化了上述四个功能，这所有的特点均使得 Shiro 更加灵活易用^[37,38]。

2.5 本章小结

本章阐述了校园轻博客系统开发中应用的关键技术，包括 Web 开发使用的 MVC 模式，以及系统的主要框架搭建框架技术 Spring Boot 和持久化框架 MyBatis，还有负责系统安全的 Apache Shiro 框架。这几大主要技术加上负责数据库的 MySQL 等其他系统相关技术会在后续实现中说明。

3 系统分析与设计

本章将分别阐述校园轻博客系统的功能、性能和安全的需求分析，系统的总体架构设计、功能设计，并根据功能进行具体设计，以及系统的数据库设计和安全设计等内容。

3.1 系统需求分析

需求分析阶段是系统开发中最首要的阶段，需求分析阶段得到的结果是项目开发中设计阶段、编码阶段、测试阶段和维护阶段的重要基础^[39]。通过对系统的需求描述分析得出系统的基本功能模块，进而得到用户角色和相关用例分析，以及与系统密切相关的性能和安全需求分析。

3.1.1 系统需求概述

校园轻博客系统主要是用于校园里学生表达自我和兴趣交流的一种平台，一方面可以发表自己的博文，记录校园生活；一方面可以通过别人发的博文发掘兴趣相同点，如读书、旅行、运动等等，进而形成自己的“小团队”，一起组织线下活动。

系统对外展示的功能应至少具备用户注册、用户登录等基本功能，同时不管作为已注册或者未注册的用户，均应具备基本的浏览博文和搜索博文的功能，作为已注册的用户，可以登录系统，同时具备修改个人信息，如密码、头像等功能，并且最重要的是对于自己的博文应具备发布、修改和删除等功能，同时为加强用户之间的互动，应对博文加上评论和点赞等互动型功能。

系统管理人员，作为后台管理人员，对系统的管理具有最高权限，对所有界面上体现出来的功能都具有操作的权限，同样包括一般用户所不具备的功能，比如直接对用户账号具有修改和注销的权限，同时可以对一些违反网络规范的博文等信息进行屏蔽和删除的权限等，以及对系统的日志、流量等监控信息进行管理。

与学校业务相关的系统虽多以功能性为主，但性能和安全性同样重要，如至少要保证系统的可靠性，更何况轻博客多以发表博文为主，这是一个可连续且频繁发生

的行为；如易用性，要求系统操作简单、方便使用，不至于使得用户在“入口”就直接失去兴趣了，而且轻博客需要用户的积极参与，否则当维护成本大于“盈利”成本时，系统就该关闭了；如可维护性，当系统出现问题时，系统维护人员需要能够尽快地定位问题、分析问题和解决问题，毕竟人们对于系统崩溃等问题地“容忍度”并不高；系统应用于学校，包含大量学生以及老师私人信息，应做好相应的安全工作。

3.1.2 系统功能需求分析

从上述的需求描述中，我们可以大概分析出校园轻博客系统的主要功能有用户信息管理、博文管理、标签管理、互动管理以及系统管理等功能。我们从系统的大致使用描述来说明这五大功能模块：

（1）用户信息管理：进入系统主界面，执行用户注册登录操作之后，用户可以通过个人信息设置来浏览和修改用户的个人信息；用户可以查询其他用户信息；可以查看通知信息，包括系统通知以及个人通知。

（2）博文管理：进入系统主界面，用户可查看当前最新更新的博文，可根据用户名或标签等条件推荐或屏蔽相关博文；用户可通过标签、用户名等多类关键字对系统进行相关内容查询；用户可编辑自己的博文，可以添加图片、文字、链接等内容，可根据自己喜好自由编辑博文格式，博文发布之后仍然保留修改博文、删除博文等操作。

（3）标签管理：进入系统主界面，用户可查看现有标签，以及标签对应的最新的博文；用户可查看自己的标签，以及对应博文；用户可以创建自己的标签，用于标记博文；用户可以查询当前标签；用户可以仅修改当前博文的标签或者修改所有使用这一标签的博文的标签；当且仅当没有任何一篇博文引用当前标签时，用户可以删除此标签。

（4）互动管理：进入系统主界面，互动操作只属于已登录用户的权限。用户可以对自己欣赏的博文进行评论、喜欢以及推荐等互动操作，对不符合网络规范的博文进行举报等操作；用户可以对自己欣赏的博主进行关注等互动操作，对自己不喜欢的博主进行拉黑等操作；管理员可以对优秀博主进行推荐、标记达人等操作，对网络行为不符合网络规范的博主进行警告、封号等相关处理。

(5) 系统管理：进入系统主界面，用户可以进行注册、登录、找回密码和注销账户等操作；还可以对于系统后台的日志、浏览量等系统信息进行记录和导出以作维护使用。

3.1.3 系统用户及用例分析

从上述的需求描述中，我们可以大概分析出校园轻博客系统的主要用户角色，主要分为两类：一类为普通用户，即使用系统的人员，主要包括学生、老师等在校人员；一类为管理员，即管理系统的人员，一般由学校网络管理人员和老师共同负责，老师负责内容的审核，网络管理人员主要负责维护系统正常运行。游客主要执行浏览和搜索的操作，而这两大块操作在普通用户的操作中均有，所以不重复说明。

用户角色确定，则用户角色对应的用例分析也随之确定：

1) 管理员

系统管理员虽对系统的所有功能都具备操作权限，但本质上只执行与网络信息审计相关的操作，对于不符合网络规范的相关内容进行检查和删除等操作，以保证网络信息内容的合法性、健康性和安全性^[40]。据此分析，以管理员为主要活动者的用例图如图 3.1 所示。

以管理员为主要活动者的用例说明：

(1) 系统通知管理，管理系统的通知发布，以及对用户发布警告通知。

(2) 用户审计，对注册用户信息进行检查：非校园网用户则拒绝注册；对违反网络规范的用户进行警告或封号处理。

(3) 博文审计，对发布的博文进行网络信息审计：管理员对违反网络规范的博文发布系统通知，通知用户进行删除或修改等操作；管理员对被举报的博文进行核实，并对违规博文和博主进行警告等操作。

(4) 标签审计，对发布的标签进行网络信息审计：管理员对违反网络规范的标签发布系统通知，通知用户进行删除或修改等操作。

(5) 互动审计，主要针对评论信息进行审计：管理员对违反网络规范的评论发布系统通知，通知用户进行删除或修改等操作。

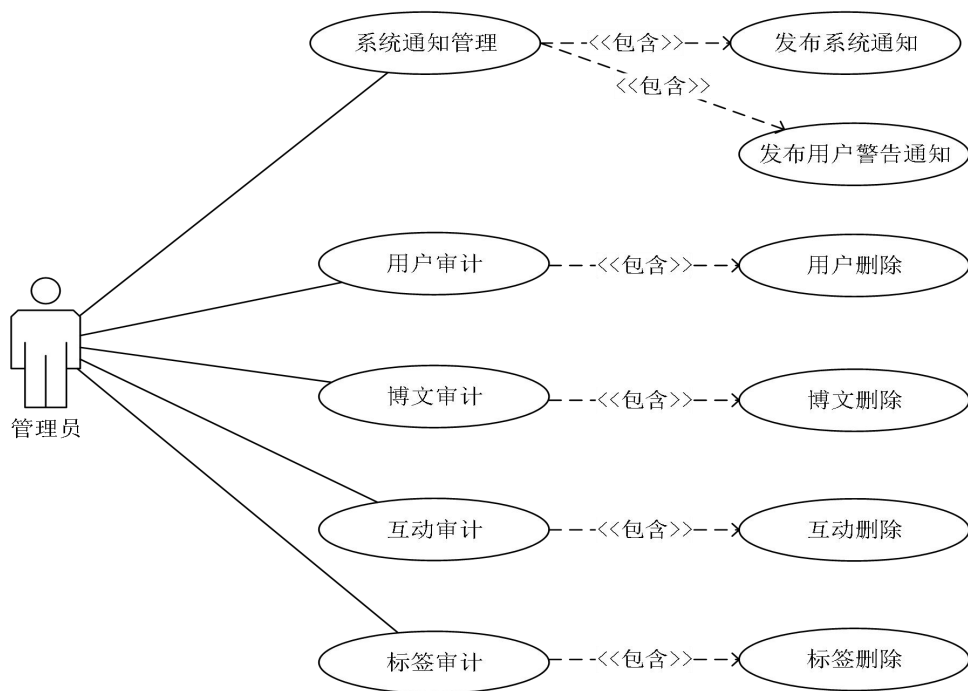


图 3.1 管理员的 UML 用例图

2) 普通用户

普通用户具有界面上显示的功能的执行权限，如用户注册登录、博文发布、评论推荐、根据标签查看博文等功能，其用例的具体描述如图 3.2 所示。

以普通用户为主要活动者的用例说明：

(1) 系统管理：管理系统操作，对账号以及私人信息（如密码）进行相关操作：用户注册增加账号，用户登录验证账号密码，找回密码更新密码等；

(2) 用户信息管理：管理用户基本信息操作，用户对其基本信息进行查看和修改等操作；

(3) 博文管理：管理博文操作，对博文进行编辑、浏览、修改、删除等操作；

(4) 标签管理：管理标签操作，标签随博文的编辑而创建或直接引用，并且可查看所有标签；

(5) 互动管理：管理用户之间的互动操作，对博文进行评论、喜欢推荐或举报，对用户进行加入黑名单等操作。

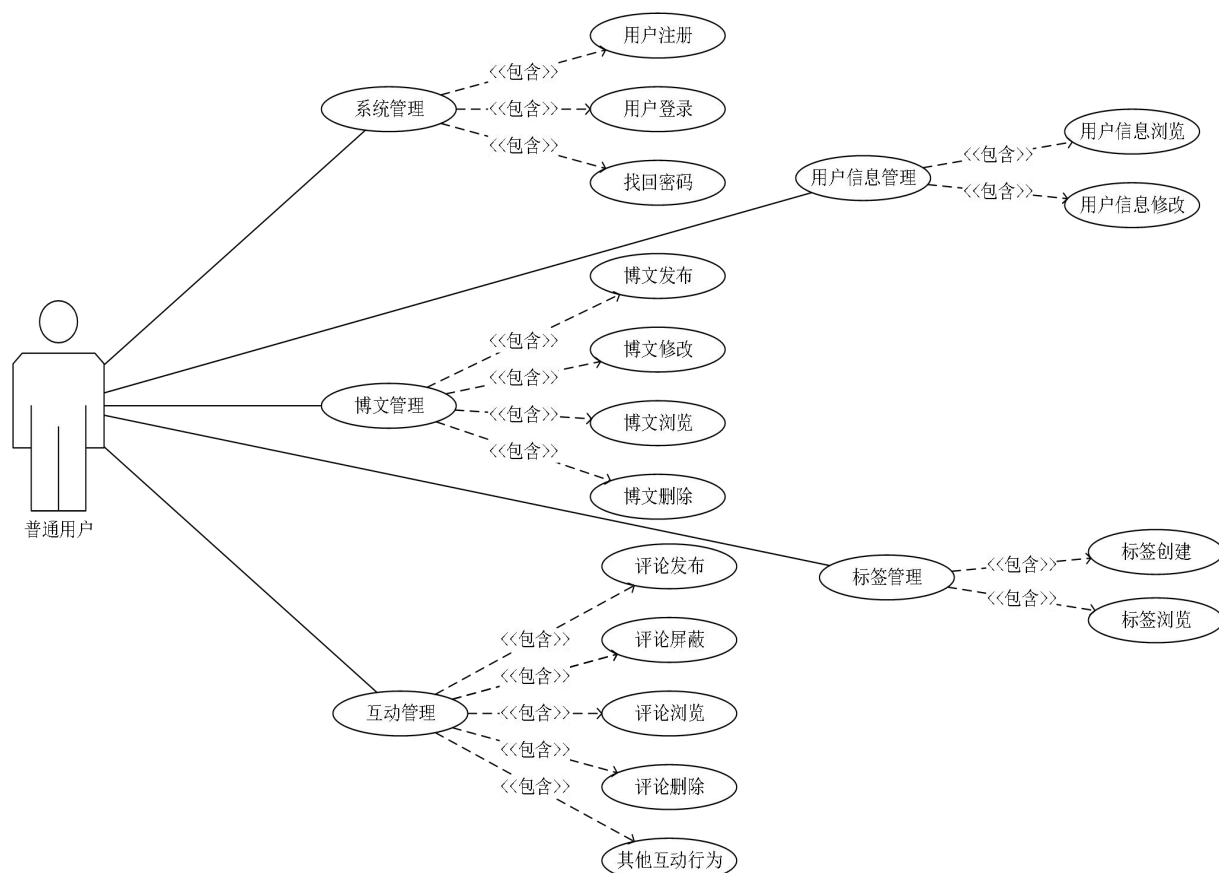


图 3.2 普通用户的 UML 用例图

3.1.4 系统性能需求分析

从狭义上讲，软件性能意味着软件使用尽可能少的系统资源的同时尽可能提高运行速度；从广义上讲，软件性能是指软件质量的属性，也就是软件的非功能性。因此在此节中性能需求分析即是非功能性需求分析，同时因系统安全性需单独说明，则将安全性需求从此节中分离，单独说明。

在系统工程及需求工程中，非功能性需求是在一定条件下系统的运行状况以及系统特性需要达到某些普遍或特定的要求^[41]。非功能性需求一般包括执行质量和发展质量两类，执行质量是指在系统运行时所具备的特性，如易用性；发展质量是指在开发过程中软件系统结构所具备的特性，如可扩展性，可测试性等^[42,43]。

校园轻博客系统主要面向学校师生，从执行质量来看，要求系统具有良好性能、易用性以及可靠性；从发展质量来看，要求系统具有可扩展性和可维护性。系统性能需求具体要求如表 3.1 所示。

表 3.1 系统性能特性需求说明表

特性名称	需求说明
性能	主要针对响应时间说明，应符合普遍的标准——2/5/10 原则：当用户在 2 秒以内得到响应，则认为系统的响应速度很快； 当用户在 2-5 秒之间得到响应，则认为系统的响应速度可以接受； 当用户在 5-10 秒以内得到响应，则认为系统的响应速度很慢，但是仍在可接受的范围内； 当用户在超过 10 秒后仍然无法得到响应时，大多数用户则认为当前请求失败，并认为系统很糟糕。 当并发操作用户量达到 8000 时，操作的响应时间应至少小于 10 秒
易用性	系统界面简洁，功能结构清晰，设置合理的导航，保证交互的流畅性，清晰的视觉层次，规则的版面等。
可靠性	在系统的使用过程中不会出现系统频繁崩溃，数据大量丢失或错乱，不会出现界面长时间的混乱，乱码等，不会出现严重的安全性问题。并且在系统周期时间（可根据具体情况决定）的使用过程中，系统正常运行的时间与总运行时间之比应保证在 99% 之上。
可扩展性	系统应支持二次开发以满足未来的业务需求，系统架构设计和代码结构应保证其松耦合性，以保证当系统需要增加新的模块时对原有系统的结构和代码不会有太大的修改之处。
可维护性	系统应支持定期数据备份和数据及时恢复，以保证系统崩溃后能尽快恢复系统正常运行。

3.1.5 系统安全需求分析

随着 Internet 的飞速发展，Web 技术的高速演变，使得 Web 服务越来越频繁地用于集成 Web 应用或与其交互^[44]。而同时段系统出现网络攻击和破坏事件也在频繁爆发，翻阅第 42 次《中国互联网络发展状况统计报告》，我们可看到 54% 的网民表示在过去半年中（2018 年 1-6 月）曾遇到过网络安全问题（个人信息泄漏和账号密码被盗），较 2017 年末增加 1.4 个百分点；2018 年上半年 CNCERT 共监测发现我国境内感染网络病毒终端累计 483 万个，被篡改的网站多达 15672 个，被植入后门的网站多达 16210 个，网站的仿冒页面多达 19060 个；2018 年上半年 CNCERT 接到的网络安全事件报告累计 54190 件，较 2017 年同期增长 12.2%，全国各级网络举报有效举报 3902.8 万件，较 2017 年同期增长 117.1%。报告中的这一项项数据足以说明系统安全性的必要性和重要性。

校园轻博客系统包含了大量在校人员信息，所以系统建立安全保障也是非常必要的，系统应具备的安全性需求为：

- (1) 对数据库操作需要进行有效隔绝，不能仅通过界面或前端提交数据即可越过后端处理而直接操作数据库；
- (2) 对提交数据需要进行有效验证，不能让错误数据没有验证就提交成功；
- (3) 对于提交后的数据需要进行过滤和修饰，再进行下一步的处理；
- (4) 对于用户传输的数据需要进行相应加密措施；
- (5) 对于用户访问需要相应的拦截控制。

3.2 系统总体设计

3.2.1 系统体系结构设计

校园轻博客系统的体系结构将采用 B/S 体系结构。B/S 体系结构由浏览器端（Browser）和服务端（Server）组成，并且是基于 C/S 体系结构发展而来的网络结构模型^[45]。在这种结构下，客户端统一使用浏览器来实现，并且浏览器端仅处理很少的业务逻辑，主要负责数据的传送和显示，绝大多数的业务逻辑将由服务端来具体实现。这样的结构使得用户不必再负载多余的客户端程序，开发人员和维护人员的工作量以及相应成本也就随之降低，进而降低了总成本^[46]。

从系统架构上来看，项目可以被分为 IE/Chrome 浏览器层、Tomcat Web 服务器层和 MySQL 数据库服务器层，如图 3.3 所示。

IE/Chrome 浏览器层是架构中直接与用户有交互行为的一层，用户通过客户端浏览器 IE/Chrome 来访问和操作系统；系统借助 Thymeleaf 模板引擎以及 Bootstrap、JQuery 等设计界面显示和部分互动；

Tomcat Web 服务器层为整个架构中的核心层，将 Tomcat 集成于 Web 容器中，为浏览器层提供服务实现；系统借助 B/S 三层结构——表示层（UI）、业务逻辑层（BLL）、数据访问层（DAL）设计和部署功能业务，其中包括用户信息管理、博文管理、标签管理、互动管理和系统管理；

MySQL 数据库服务器层是架构中的最底层，也是很重要的一层，因为当今时代正是一个数据“无价”的时代，这一层主要集成了 MySQL 数据库的相关服务，为 Tomcat Web 服务器层提供数据存储和数据查询等服务。

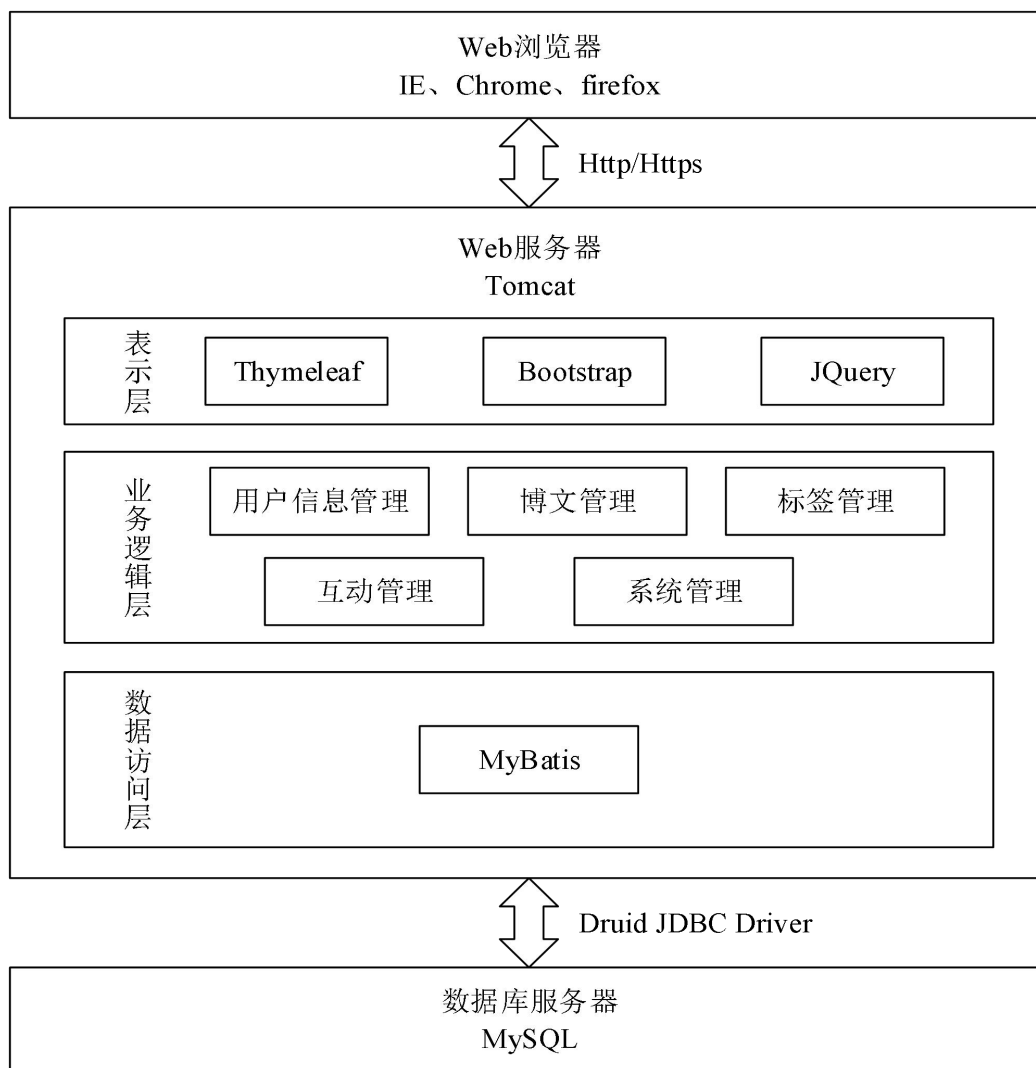


图 3.3 系统架构图

3.2.2 系统功能结构设计

从 3.1.2 系统功能分析与 3.1.3 系统用户以及用例分析两个小节中可以分析得到校园轻博客系统的总体功能结构，其中包括核心功能模块：对于博文、标签的管理模块，用户之间或者用户与博文之间互动的管理模块，以及对于个人信息和通知的管理模块和系统操作的管理模块，如图 3.4 所示。每个模块的具体功能结构将在其模块的具体设计中说明。

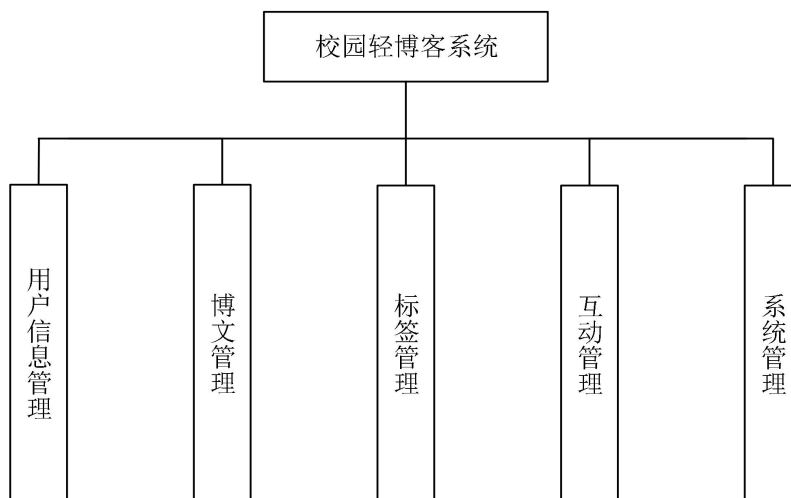


图 3.4 系统总体功能结构图

3.3 系统功能模块设计

校园轻博客系统共包含五大模块：用户信息管理模块、博文管理模块、标签管理模块、互动管理模块以及系统管理模块。下面将对这五大模块的具体功能进行说明。

3.3.1 用户信息管理模块

用户信息管理模块主要是针对用户信息以及通知信息的管理模块，主要包括用户信息浏览、用户信息修改、用户搜索、通知信息浏览、通知信息修改，功能结构如图 3.5 所示。

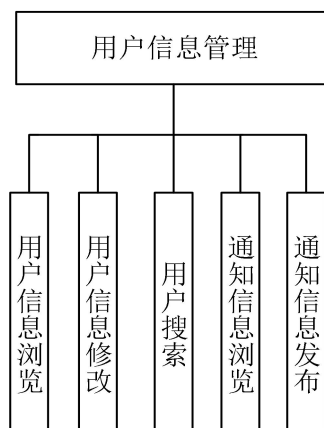


图 3.5 用户信息管理模块的功能结构图

（1）用户信息浏览

此功能方便用户在登录之后查看个人信息。这里主要涉及到的是从数据库中根据条件获取用户基本信息，并展示在页面上的操作。

（2）用户信息修改

此功能在用户查看个人信息后，对基本信息和个人主页的状态信息进行修改。这里主要涉及到的是从数据库中获取以前的用户信息，并根据用户输入的修改后的信息进行数据更新操作，并刷新用户信息浏览界面的操作。

（3）用户搜索

此功能可以通过用户名模糊查询当前已注册用户，主要是管理员在后台进行用户管理的操作。这里主要涉及到的是从数据库中根据条件获取用户对象信息，并将所有匹配到的结果展示在页面上的操作。

（4）通知信息浏览

此功能方便用户登录之后，收到来自系统或者他人的信息时，可以进行查看相关信息细节。这里主要涉及到的从数据库中根据条件获取通知信息，并将通知的详细信息展示在页面上的操作。

（5）通知信息发布

此功能主要针对管理员，主要功能是管理员发布系统通知，或者对无法通过内容审计的已注册的博主发布警告通知。这里主要涉及到的通知信息的创建，并且存储到数据库中，同时被通知的对应用户应收到相应消息提醒等操作。

用户信息的浏览和搜索总是联系在一起，而通知浏览主要只是查看功能，通知发布只能由管理员实现。这里仅以用户信息修改作为说明，用户基本信息包括用户姓名、邮箱、描述、头像和密码，这里为了将密码这类私密信息以及头像这类非字符串格式的信息与其他字符串格式的信息区分开来，系统将用户基本信息分为三类：一类为姓名、邮箱和描述，一类为头像，一类为密码。这三类信息修改的流程基本一致，所以这里抽象成一种，如图 3.6 所示。

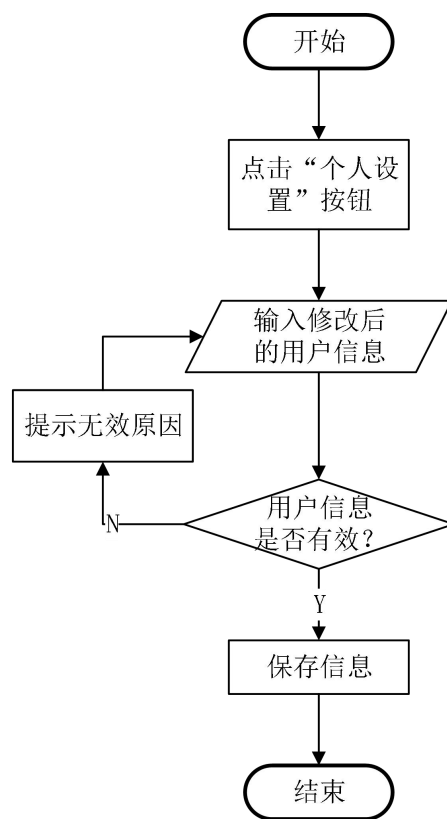


图 3.6 用户信息修改功能的流程图

3.3.2 博文管理模块

博文管理模块主要针对博文内容相关操作的管理,主要包括博文发布、博文浏览、博文修改、博文删除、博文搜索,其结构如图 3.7 所示。

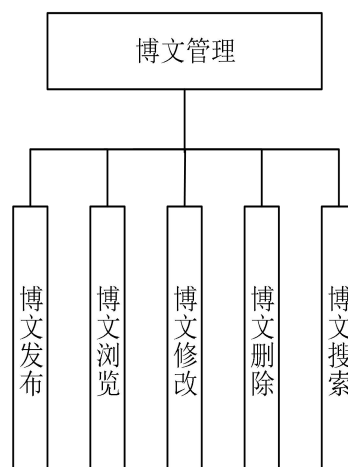


图 3.7 博文管理模块的功能结构图

（1）博文发布

此功能方便用户在登录之后对文字、图片、链接等内容自由编辑，系统还将支持 Markdown 编辑方式，以满足不同需求。轻博客鼓励为博文打上标签，方便“同类”查询，以获得更多相关信息，编辑完成即可发布，可设置是否允许公开以及是否允许评论等信息。这里主要涉及到的是博文的创建、标签的查询或创建，以及将博文和博文标签关系存储到数据库，并且刷新用户博文界面等操作。

（2）博文浏览

此功能方便用户查看自己或者他人的博文，可通过关注的用户或标签等信息进行分类浏览博文。这里主要涉及到的是从数据库中根据条件获取博文信息，并将博文基本信息、与博文相关的博主信息以及像评论、喜欢和推荐等互动信息在界面上展示出来等操作。

（3）博文修改

此功能用于在用户登录之后，用户可以对博文的内容和格式等进行修改操作，同时也可以修改博文查看的权限。这里主要涉及到的是从数据库中获取博文信息，并根据用户提交的修改后的信息进行数据更新，并且将最新博文信息以及与博文相关的最新信息展示在界面上等操作。

（4）博文删除

此功能用于在用户登录之后，用户可以对已存在的自己的博文进行删除操作；当管理员审计用户博文，出现不符合网络规范的博文时，管理员有直接删除用户博文的权限。这里主要涉及到的是对数据库中的博文信息以及与博文相关的所有信息进行删除，并且刷新当前用户博客展示界面等操作。

（5）博文搜索

此功能方便用户通过博文名对相关博文进行模糊查询操作。这里主要涉及到根据条件从数据库中获取博文信息，并且在界面上展示出所有相关结果等操作。

博文修改与博文创建只多了一个验证博客是否存在，以及获取博客原来内容的操作，其后操作与博文创建差不多，而博文浏览与博文搜索常常关联，博文删除只需检查博文是否存在，博文的 owner 是不是当前用户或者管理员即可，这里仅以博文创建的过程作为说明，其创建流程如图 3.8 所示。

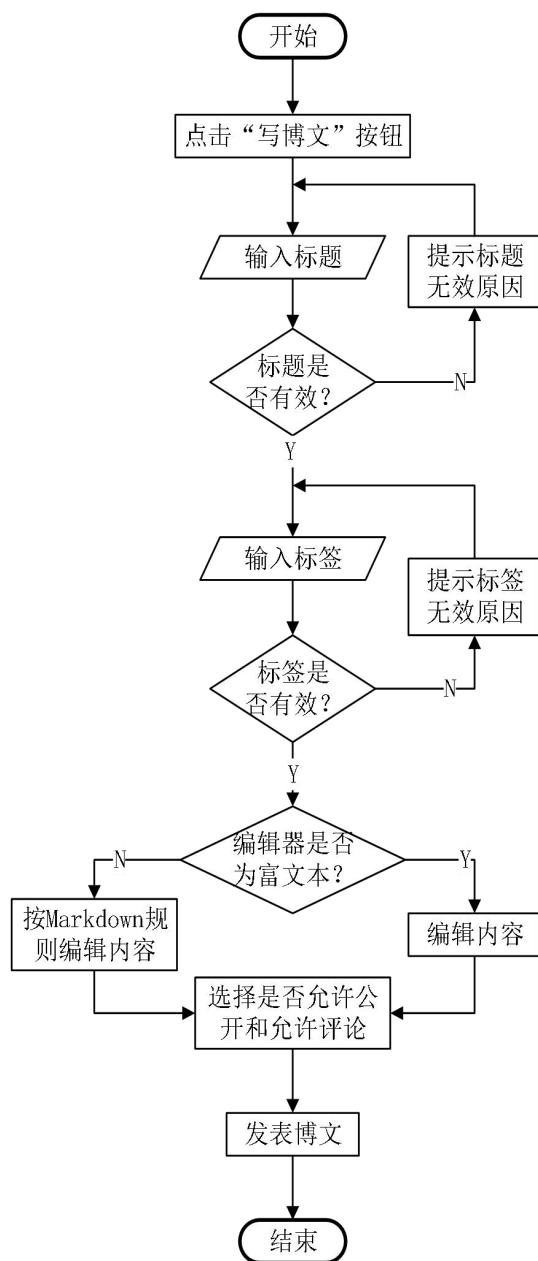


图 3.8 博文创建功能的流程图

3.3.3 标签管理模块

标签管理模块主要是针对标签的增删改查操作，主要包括标签创建、标签查看、标签修改、标签删除、标签搜索等功能模块。对于管理员而言，这几大功能均具备；对于一般用户，仅具有标签创建和标签搜索查看等功能，并且标签的创建随博文的创建进行。模块的功能结构如图 3.9 所示。

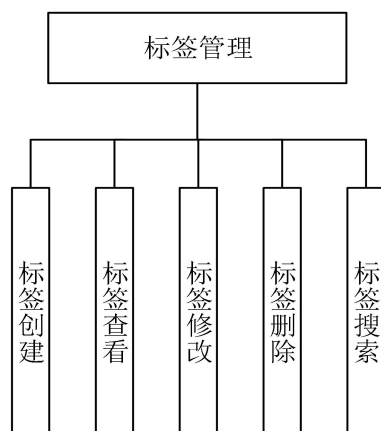


图 3.9 标签管理模块的功能结构图

（1）标签创建

此功能方便用户在登录之后创建自己的个性标签，并通过标签进一步挖掘自己的兴趣爱好。这里主要涉及到的是标签的创建，将新标签存储到数据库中，并刷新标签显示界面等操作。

（2）标签查看

此功能方便用户查看自己或者他人的标签，以及此标签标记的相关博文信息。这里主要涉及到的是从数据库中根据条件获取标签信息，并将标签信息展示在结果页面上等操作。

（3）标签修改

此功能用于在用户（管理员）登录之后，用户可以对标签进行修改操作。这里主要涉及到的是从数据库中获取标签信息，并且根据用户提交的信息进行数据更新，并且刷新当前标签展示页面等操作。

（4）标签删除

此功能用于在用户（管理员）登录之后，用户可以对已存在的标签进行删除操作；当管理员审计用户标签，出现不符合网络规范的标签时，管理员有直接删除用户标签以及相关博文的权限。这里主要涉及到的是对与标签相关的信息进行检查操作，然后对数据库中的标签信息进行删除，并且刷新当前标签展示页面等操作。

（5）标签搜索

此功能方便用户通过标签名对相关标签及其标记的博文进行搜索查看。这里主要

涉及到的是根据条件从数据库中获取标签基本信息以及彼此标签标记的博文基本信息，并将所有信息显示在查询结果页面上等操作。

对于普通用户来说，标签模块与博文模块紧密相连：标签的创建是随着博文创建过程中填写相关标签所引起的操作；标签的查找和查看是借助标签查看博文集的过程；标签的修改和删除都需要对相关的博文进行对应的修改的操作。这里单独说明标签的创建过程，其流程图如图 3.10 所示。

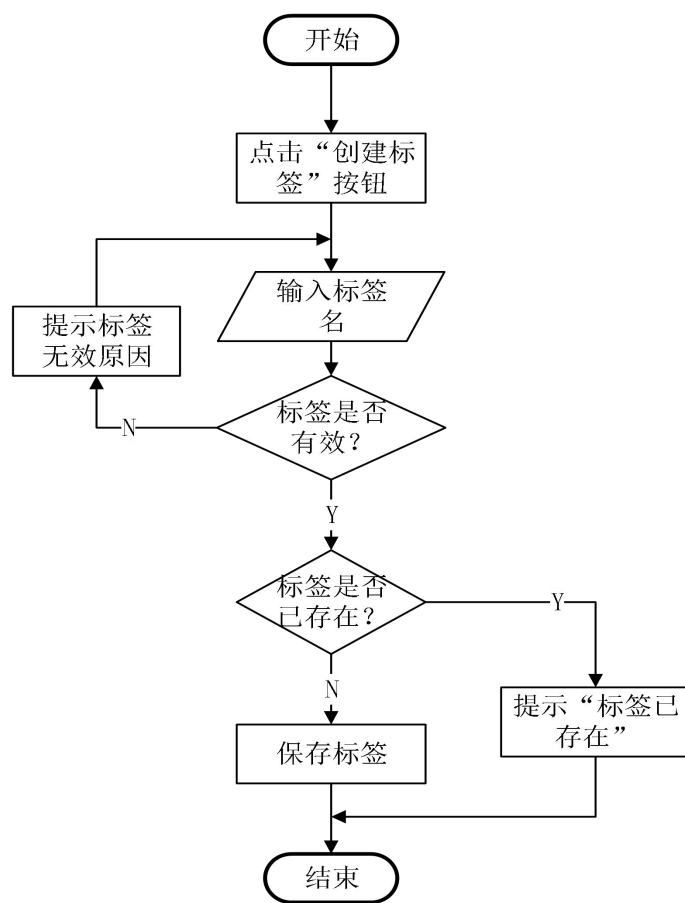


图 3.10 标签创建功能的流程图

3.3.4 互动管理模块

互动管理模块主要是针对用户与博文之间以及用户与用户之间的互动操作进行管理，如评论、喜欢、推荐等互动操作，还包括黑名单、屏蔽、举报等帮助维护良好网络环境的互动操作，其功能结构图如图 3.11 所示。

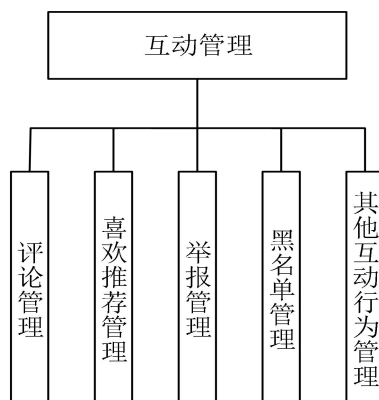


图 3.11 互动管理模块的功能结构图

（1）评论管理

此功能方便用户对其他人的博文发表友好评论，用户可以查看博文评论，对评论进行回复，可以对自己博文下的不好评论进行删除操作。这里主要涉及到的是对评论的增删改查操作。此模块需要注意的是用户对博文进行评论操作和用户评论进行回复操作至少涉及了用户和博文两类对象，在对数据库进行操作之前需要先完成关联关系的补充或者删除操作。

（2）喜欢推荐管理

此功能方便用户对自己喜欢的博文进行喜欢和推荐操作，用户在浏览博文时不仅可以看到关注者所发的博客，也可以看到关注者所推荐的博客。这里主要涉及到的是对喜欢和推荐的增删查操作。此模块同样涉及到用户和博文两类对象，需要对关联关系进行处理后再进行后续操作。

（3）举报管理

此功能方便用户对不符合网络规范的内容进行举报，以维护网络“卫生”。这里主要涉及到的是对举报内容的增删查操作。此模块也涉及到用户和博文两类对象，需要处理关联对象信息后再处理自身信息的存储和删除。

（4）黑名单管理

此功能用于在用户登录之后，用户可以对自己不喜欢的博主进行拉黑操作，即将相关博主加入黑名单，以屏蔽相关博主的信息。这里主要涉及到的是对黑名单信息的增删查操作。此模块主要涉及到主动拉黑用户和被拉黑用户对象，同样需要先处

理关联关系再进行自身信息的编辑。

(5) 其他互动行为管理

此功能主要针对除上述之外的互动性行为的管理。

这五种互动操作的流程基本一致，都是检查用户是否已经登录，以及互动的对象是否有效，再进行互动操作，这里仅以评论功能的流程图作为代表进行说明，用户评论功能的流程图如图 3.12 所示。

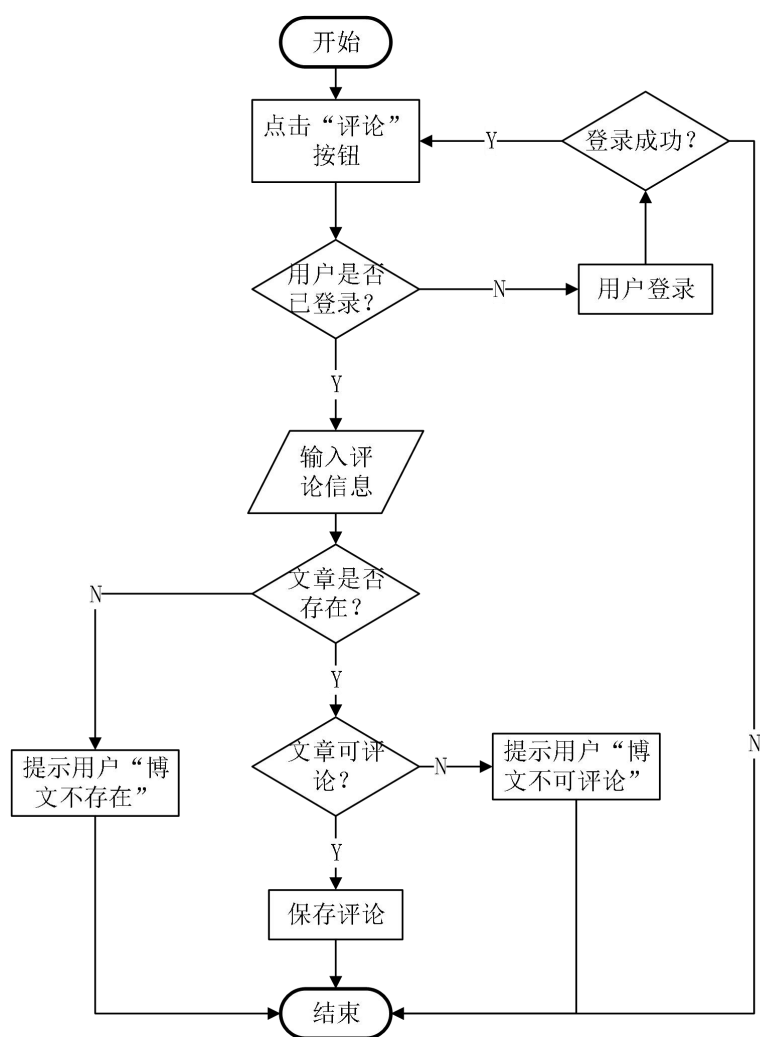


图 3.12 用户评论功能的流程图

3.3.5 系统管理模块

系统管理模块主要处理系统的用户操作，如用户登录、用户注册、找回密码以及用户注销等操作，其模块功能结构图如图 3.13 所示。

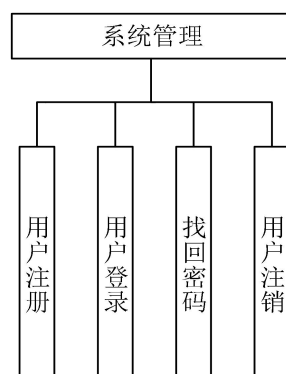


图 3.13 系统管理模块的功能结构图

（1）用户注册

此功能主要实现用户注册，非学校人员不能注册。这里主要涉及到根据条件从数据库中查询用户对象信息，并在没有的前提下创建用户，并将用户基本信息存储到数据库中，然后刷新当前用户界面等操作。

（2）用户登录

此功能主要实现用户登录，采用用户名+密码的形式登录。这里主要涉及到根据条件从数据库中获取用户信息，在存在的前提下验证密码正确性，并在界面上显示反馈信息等操作。

（3）找回密码

此功能方便用户通过邮箱验证找回系统密码。这里主要涉及到根据条件从数据库中获取用户信息，在存在的前提下向用户发送验证邮件，根据邮件中的链接进入密码修改界面，将用户提交的新密码更新到数据库中，并转至登录界面等操作。

（4）用户注销

此功能帮助用户在不想使用系统之后销毁账户以及博文等信息。在校人员离开学校时，可以注销账号以及信息，或者将博文打包保存本地。这里主要涉及到从数据库中获取账户信息，对账户以及与账户相关的博文、互动等信息进行删除，并且转至系统未登录时的首页等操作。

用户登录、找回密码，以及用户注销都必须保证用户已经注册过了，所以用户注册是所有需要用户 ID 的功能的基础功能，所以这里主要介绍用户注册的工作流程，如图 3.14 所示。

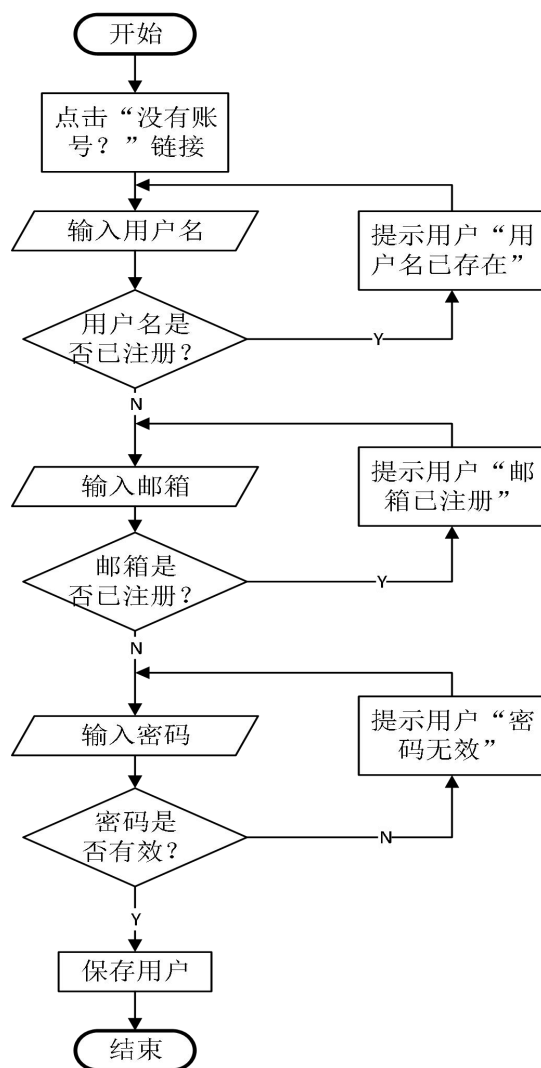


图 3.14 用户注册功能的流程图

3.4 系统数据库设计

数据库设计总体来说，包含四大阶段：需求分析阶段、概念设计阶段、逻辑设计阶段以及物理设计阶段。需求分析阶段在 3.1 和 3.2 节已详细说明，概念设计阶段会根据需求分析得到的结果得出独立于系统的概念模型，逻辑设计阶段是将概念模型转换成特定数据库的数据模型结构，最后物理设计阶段则是根据具体数据库语言设计物理存储结构，建立数据库和相应表^[47]。

系统将借助数据库设计工具 PowerDesigner 对数据库建模。通过对需求分析得到的结果进一步分析得到模块的具体属性以及模块与模块之间的关系模型，然后借助

PowerDesigner 生成逻辑模型以及物理模型，最后生成数据库 SQL 执行文件，得到对应 MySQL 数据库的表结构。

3.4.1 数据库建模说明

根据本章 3.1-3.3 节的需求分析说明，系统将涉及到三个主要实体，分别是用户、博文以及标签；多个与互动相关的实体，分别是评论、喜欢、推荐，以及关注、通知、黑名单。

实体之间的关系分为四种：一对一、一对多、多对一以及多对多。

1) 首先说明三个主要实体之间的关系：

(1) 用户与博文的关系为一对多：一个用户可以拥有多篇博文，而一篇博文只会有一个用户（作者）；

(2) 博文与标签的关系为多对多：一篇博文可以拥有多个标签，而一个标签也可以标记多篇博文；

2) 然后说明在用户实体之间的互动操作关系：

(1) 通知：一个用户可以收到多条通知；而一个用户（管理员）可以发出多条通知；

(2) 黑名单：一个用户可以将多个用户加入黑名单，一个用户也可以被多个用户加入黑名单；

(3) 关注：一个用户可以关注多个用户，一个用户也可以被多个用户关注；

3) 接着说明在用户实体和博文实体之间的互动操作关系：

(1) 评论：一个用户可以评论多篇博客，一篇博客也可以被多个用户评论；

(2) 喜欢：一个用户可以喜欢多篇博客，一篇博客也可以被多个用户喜欢；

(3) 推荐：一个用户可以推荐多篇博客，一篇博客也可以被多个用户推荐；

4) 最后说明回复评论与评论以及用户之间的关系：

(1) 评论：一个用户可以对多个评论进行回复，一个评论也可以被多个用户回复；

(2) 回复评论：一个用户可以对多个回复进行再一次的回复，一个回复也可以被多个用户进行再一次的回复。

根据上述分析可以得到校园轻博客系统数据库的实体关系（E-R）图，如图 3.15 所示。

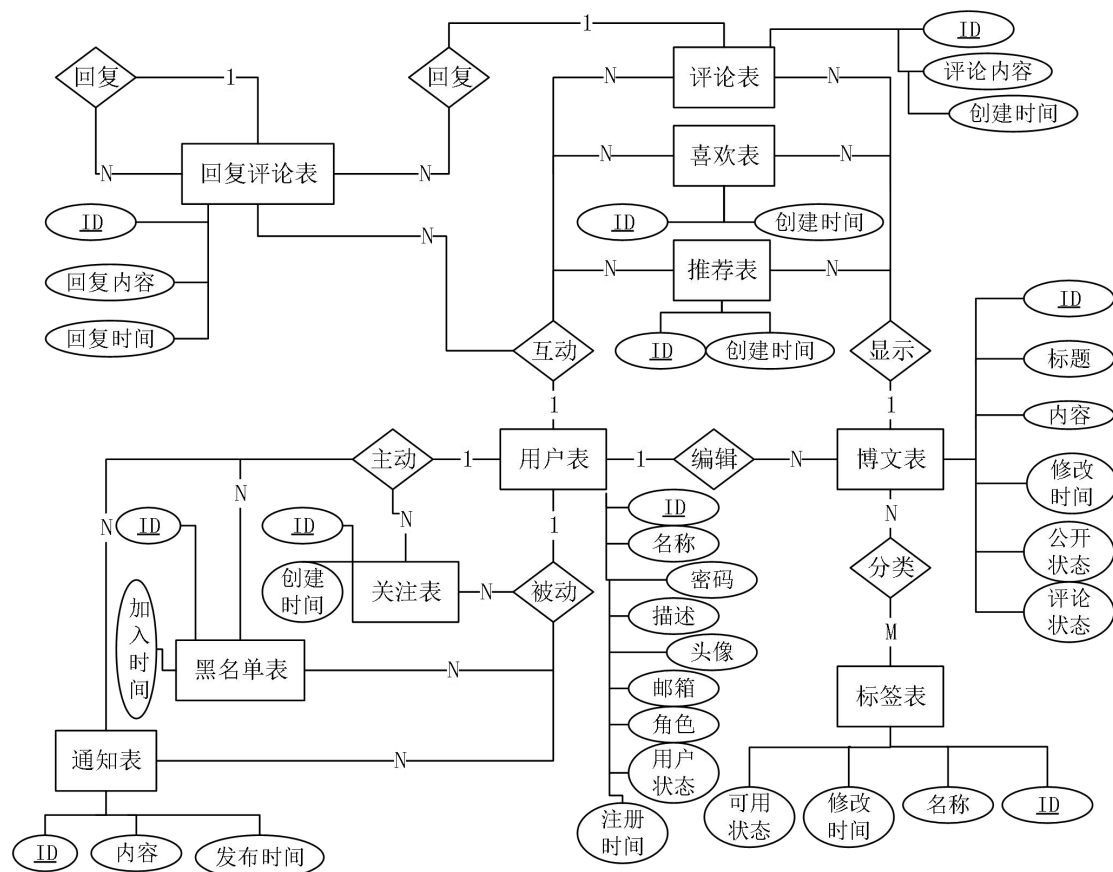


图 3.15 数据库 E-R 图

3.4.2 数据库表结构说明

根据上述分析，除了博文与标签之间多对多的关系会生成一个额外的关系表，其他实体只对应生成一张表，但部分表因为存在一对多的关联关系，所以在表内会产生相应外键。系统共产生 11 张表，PowerDesigner 工具根据绘制的系统逻辑模型以及选择的数据库类型等参数生成对应物理模型，最后生成 SQL 执行文件，并且利用 Navicate 工具在 MySQL 数据库中生成对应表。

(1) 用户表结构

用户表包含用户 ID、用户昵称、密码、描述、头像、邮箱、角色、可用状态和注册时间属性，如表 3.2 所示。

华中科技大学硕士学位论文

表 3.2 用户表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
UID	int(11)	primary key	用户 ID
UName	varchar(64)	Not null	用户昵称
UPassword	varchar(64)	Not null	密码
UDescription	text		描述
UPicture	longblob		头像
UEmail	varchar(60)	Not null	邮箱
URole	smallint(6)	Not null	角色
UStats	smallint(6)	Not null	可用状态
URegisterTime	datetime	Not null	注册时间

(2) 博文表结构

博文表包含博客 ID、博客所属用户 ID、博客标题、博客内容、博客修改时间、博客公开状态和博客评论状态属性，如表 3.3 所示。

表 3.3 博文表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
BID	int(11)	primary key	博客 ID
UID	int(11)	foreign key	博客所属用户 ID
BTitle	char(64)	Not null	博客标题
BContent	text	Not null	博客内容
BCreateTime	datetime	Not null	博客修改的时间
BShowStatus	tinyint	Not null	博客是否公开
BCommentStatus	tinyint	Not null	博客是否可以评论

(3) 标签表结构

标签表包含标签 ID、标签名称、修改时间和可用状态属性，如表 3.4 所示。

表 3.4 标签表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
LID	int(11)	primary key	标签 ID
LName	varchar(255)	Not null	名称
LCreateTime	datetime	Not null	修改时间
LStatus	tinyint(1)	Not null	可用状态

(4) 博文标签关系表结构

博文标签关系表包含博客 ID 和标签 ID 属性，如表 3.5 所示。

表 3.5 博文标签表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
BID	int(11)	primary key	博客 ID
LID	int(11)	primary key	标签 ID

(5) 通知表结构

通知表包含通知 ID、发送通知用户 ID、接收通知用户 ID、通知内容和通知时间属性，如表 3.6 所示。

表 3.6 通知表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
NID	int(11)	primary key	通知 ID
NsUID	int(11)	foreign key	发送通知用户 ID
NrUID	int(11)	foreign key	接收通知用户 ID
NContent	text	Not null	通知内容
NTime	datetime	Not null	通知时间

(6) 黑名单表结构

黑名单包含黑名单 ID、用户 ID、被拉黑用户 ID 和操作时间属性，如表 3.7 所示。

表 3.7 黑名单表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
BKID	int(11)	primary key	黑名单 ID
BKsUID	int(11)	foreign key	用户 ID
BKrUID	int(11)	foreign key	被拉黑用户 ID
BTime	datetime	Not null	黑名单时间

(7) 关注表结构

关注表包含关注 ID、用户 ID、被关注用户 ID 和关注时间属性，如表 3.8 所示。

表 3.8 关注表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
FID	int(11)	primary key	关注 ID
FsUID	int(11)	foreign key	用户 ID
FrUID	int(11)	foreign key	被关注用户 ID
FTime	datetime	Not null	关注时间

(8) 评论表结构

评论表包含评论 ID、用户 ID、博客 ID、评论内容和评论时间属性，如表 3.9 所示。

表 3.9 评论表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
CID	int(11)	primary key	评论 ID
UID	int(11)	foreign key	用户 ID
BID	int(11)	foreign key	博客 ID
CContent	varchar(255)	Not null	评论内容
CTime	datetime	Not null	评论时间

(9) 评论回复表结构

评论回复表包含回复评论 ID、回复的父回复 ID、回复用户 ID、回复评论内容和回复评论时间属性，如表 3.10 所示。

表 3.10 回复评论表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
RCID	int(11)	primary key	回复评论 ID
ReRCID	Int(11)	foreign key	回复的父回复 ID
UID	int(11)	foreign key	回复用户 ID
RCContent	varchar(255)	Not null	回复评论内容
CTime	datetime	Not null	回复评论时间

(10) 喜欢表结构

喜欢表包含喜欢 ID、博客 ID、用户 ID 和喜欢时间属性，如表 3.11 所示。

表 3.11 喜欢表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
LiID	int(11)	primary key	喜欢 ID
BID	int(11)	foreign key	博客 ID
UID	int(11)	foreign key	用户 ID
LiTime	datetime	Not null	喜欢时间

(11) 推荐表结构

推荐表包含推荐 ID、用户 ID、博客 ID 和推荐时间属性，如表 3.12 所示。

表 3.12 推荐表结构

Name 名称	Type 类型	Restriction 限制	Description 描述
RID	int(11)	primary key	推荐 ID
UID	int(11)	foreign key	用户 ID
BID	int(11)	foreign key	博客 ID
RTime	datetime	Not null	推荐时间

3.5 系统安全设计

根据 3.1.5 系统安全需求分析，系统做了总体的安全设计：

(1) 系统采用分层设计结构，从而能保证每一层仅能与当前层或与上一层进行直接交互，意味着在三层结构中表示层不能直接操作数据访问层，即无法直接通过修改用户提交表单来操作数据库数据；

(2) 系统设置数据的多层验证，通过 Ajax 来实现表单提交的基本数据格式和简单逻辑验证，通过 Web 的 shiro 标签配置和 shiro 权限注解实现前后端的操作权限一致，在后端对提交的数据进行重复检查，以及其他业务逻辑检查，如用户身份的验证等；

(3) 系统尽量保证安全编码，对所有用户提交的表单数据进行 xss（跨站脚本攻击）过滤，对部分输入参数进行相应的安全转换，采用参数化查询，而非直接拼

接参数执行，控制数据输入范围等安全性检查；

(4) 系统为用户密码加密，将采用 2 次 MD5 散列算法对密码进行盐值加密，此过程是不可逆的过程，所以原始信息无法根据散列得到的结果被推导出来，从而保障了密码的安全传输；

(5) 系统严格控制用户访问资源，通过在访问资源之前进行用户身份和操作权限对比等相关验证操作来防止用户直接通过修改提交表单以绕过权限检查，直接进行越权操作。

这五项设计中分层结构设计主要是系统的整体架构设计，多层验证中的数据验证与前端验证设计和后端方法设计有直接关系，其他三项的设计本文中主要针对身份验证以及授权访问资源，以及 xss 内容过滤等功能进行说明。

xss 内容过滤准备借助 Filter 对请求中包含的非法标签等内容进行过滤，其大致流程如图 3.17 所示。

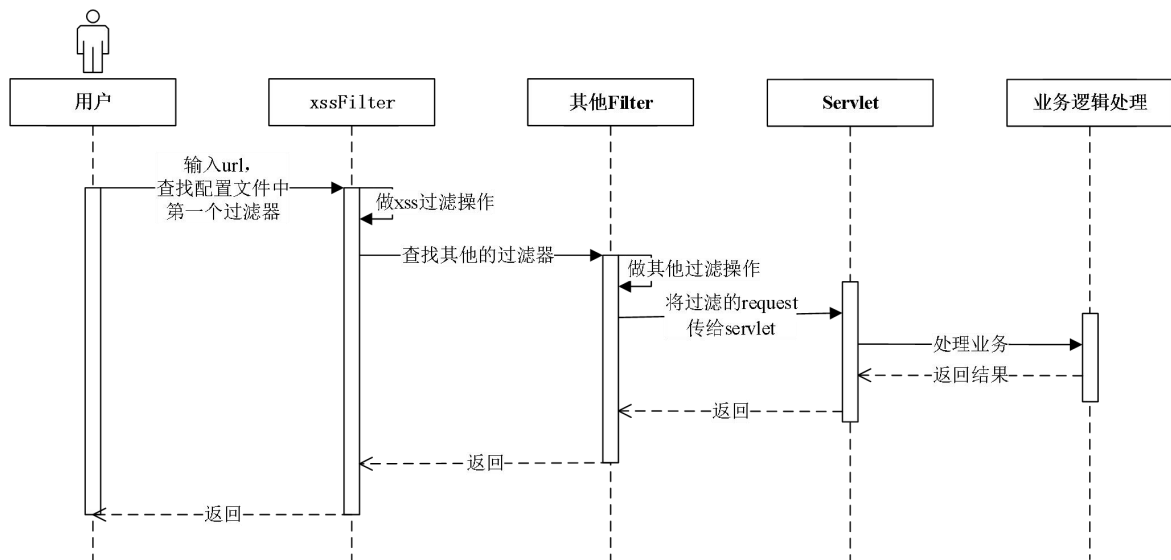


图 3.17 xss 内容过滤的时序图

身份验证和授权访问资源主要是针对用户在访问资源时对用户身份、操作权限与预存的安全数据（包括用户密码以及用户权限字符串）的对比验证，其大致流程如图 3.18 所示。

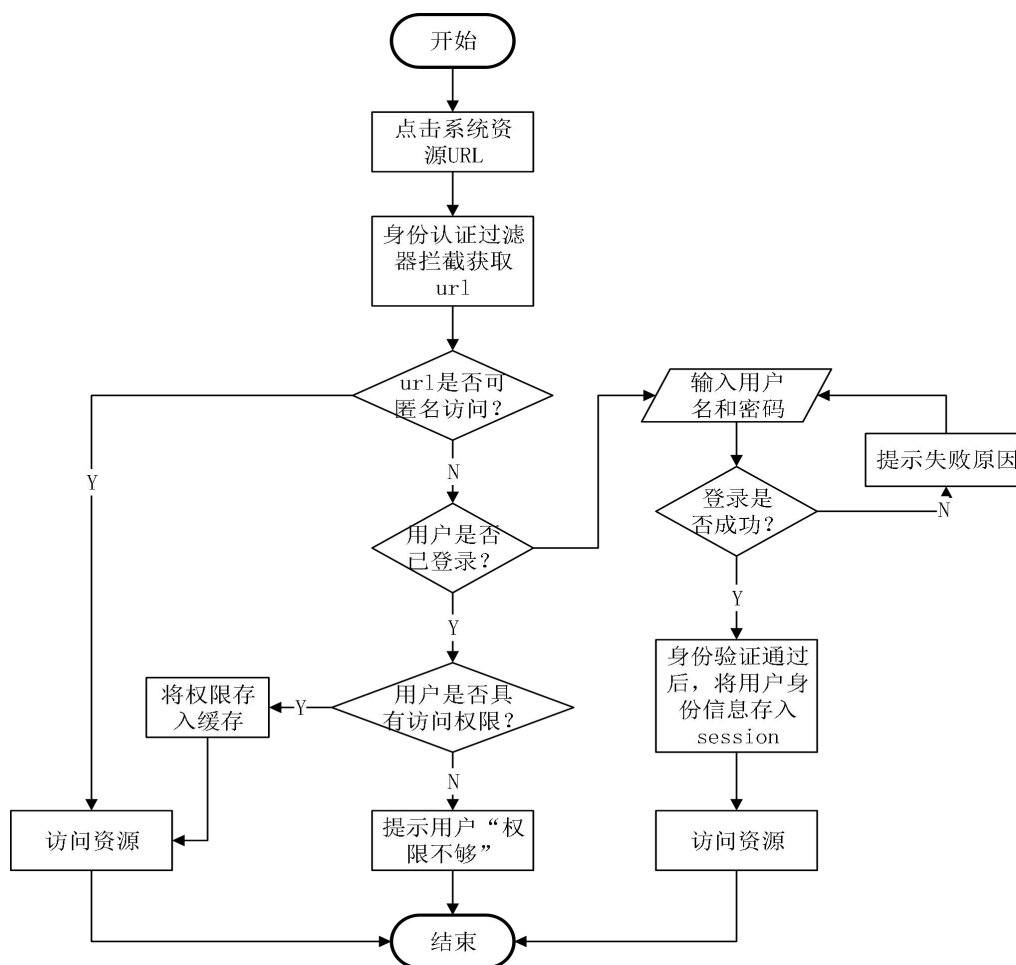


图 3.18 用户授权访问的流程图

3.6 本章小结

本章首先对校园轻博客系统的功能结构、用户角色和相关用例进行了需求分析，同时也对系统的性能和安全需求进行了分析；其次对系统整体的体系结构进行了分析，再根据需求分析的结果设计了系统的整体功能结构；再次对系统的功能模块进行了详细设计，包括对系统的功能进行说明，绘制功能结构图以及系统流程图等；最后分别对系统的数据库和安全性进行了设计，通过 PowerDesigner 绘制数据库的 E-R 图，并设计了基于 MySQL 数据库的 Blog 数据库以及它的所有实体以及实体关系表；通过系统的结构设计、各类数据验证、数据安全过滤、用户密码加密、用户权限检查等多种操作保证了系统的安全性。

4 系统实现与测试

本章将对校园轻博客系统的实现和测试过程进行详细介绍。本章将从系统实现的开发环境、系统的部分功能具体实现、系统的安全实现、系统的部分功能界面以及系统的测试等方面进行具体介绍。

4.1 系统开发环境

系统的开发实现环境包括以下几个方面：

- (1) 采用 Inter(R) Core(TM) i5-3230M 的 CPU、2.6GHz 的主频以及 4G 内存、120G 硬盘的硬件环境；
- (2) 采用 Java 面向对象编程语言作为开发语言；
- (3) 采用 IDEA 2018.1 作为开发平台，配置 JDK 1.8 保证 Java 运行环境，配置 Tomcat 9.0.8 作为 Web 服务器，选择 maven 的项目结构方便进行项目构建，依赖管理以及仓库管理等工作；
- (4) 采用 MySQL 8.0.11 作为数据库存储数据，并采用 Navicat Premium 12.0.28 作为管理数据库，并通过界面来操作数据库的工具，以达到改善工作流程并提高工作效率的目的；
- (5) 采用 Visio 2016 绘制系统的各类分析图，如 UML 用例图，流程图等；
- (6) 采用 PowerDesigner 16.5 对系统数据库建模，绘制相应概念模型、物理模型等；
- (7) 采用 Spring Boot 2.0.3 + MyBatis 3.4.4 的系统框架进行开发，在 pom 文件中引入相关依赖包；
- (8) 采用操作系统 Windows 10 以及浏览器 IE7/8、chrome、Microsoft Edge 等。

4.2 系统功能模块实现

从 3.3 系统的功能模块设计小节中可以得到系统共分为以下五大模块，分别是用户信息管理模块、博文管理模块、标签管理模块、互动管理模块和系统管理模块。

本小节将先就系统的公共部分进行说明，后仅针对各模块的功能的业务逻辑进行说明。因篇幅有限，仅介绍五大模块中的核心功能实现。

4.2.1 系统公共部分说明

系统的文件结构大致共包含 5 个包：`com.blog.notices`、`com.blog.blogs`、`com.blog.labels`、`com.blog.interactions`、`com.blog.systems`，每个包下主要包含 `controller`、`service`、`dao`、`domain` 这四大类。`controller` 包存放 `XXController` 类，负责前端与后台的交互；`service` 包存放 `XXService` 接口和它对应的 `XXServiceImpl` 实现类，负责业务逻辑；`dao` 包存放 `XXDao` 接口，负责与数据库交互；`domain` 包存放对象类，如 `User`。除这四大类以外，有的包中可能还有一些其他的类，如 `util` 包——用于放置工具类、`config` 包——用于放置配置类、`aspect` 包——用于放置切面类、`exception` 包——用于放置异常类等等。

系统采用 `thymeleaf` 模板渲染作为 `view` 部分，采用 `thymeleaf` 作为模板语言，用来接收返回的模板数据。`view` 到 `controller` 的连接需要在 `pom` 文件中引入相关依赖，在配置文件中添加相关配置，在模板中加入标签库，编辑模板视图。

系统采用大量注解，为 `controller` 类采用 `@Controller` 注解，为 `service` 类采用 `@Service` 注解，为 `dao` 类采用 `@Mapper` 注解。在 `controller` 层还采用 `@RequestMapping` 等相关注解定义请求路径和参数等数据，采用 `@RequestBody` 或 `@ResponseBody` 将数据以固定格式（`json` 格式）的获取或返回，采用 `@RequiresPermissions` 等限制访问接口的用户权限。在 `service` 层还可以采用 `@Transactional` 进行事务管理。在 `dao` 层也可采用 `@Select`、`@Insert`、`@Update` 或 `@Delete` 等注解直接在对应接口上定义 `sql` 语句，但本系统采用 `mapper.xml` 的形式定义数据库操作。

系统采用 `mybatis` 数据持久层框架和 `MySQL` 数据库，采用 `SQL` 语句的形式定义数据库中数据的 `CRUD` 操作。配置方面需在 `pom` 中引入相关依赖，需进行数据源配置以及 `mybatis` 的基本配置，统一采用 `dao` 对应的 `mapper.xml` 来编写 `SQL` 语句，并进行相关 `mapperScanner` 和 `mapping` 的配置来实现 `dao` 和 `mapper.xml` 对应关系。

对于日志和缓存等一类具有“横向”特性的功能，则利用 `spring` 的 `AOP`（面向切面编程的特性）的注解 `@Aspect`、`@Pointcut`、`@Before` 等定义相关日志操作。

4.2.2 用户信息管理模块实现

用户信息管理模块包括了用户信息浏览、用户信息修改、用户查询、通知浏览、通知发布功能，本模块中主要说明用户信息浏览和用户信息修改功能。

此模块主要涉及到的对象有 `User`、`UserController` 以及主要方法 `getUser`、`updateUser`、`deleteUser`、`addUser`。

用户信息浏览和用户信息修改功能主要说明用户个人信息的查看和修改，用户个人信息根据具体模块设计分为三类：姓名、描述、邮箱信息和头像信息以及密码信息，所以在修改功能上会根据具体参数修改对应属性，在这里主要涉及 `getUser` 和 `updateUser` 方法。具体方法说明如下：

1) 用户信息浏览功能

(1) 方法名：`getUser`;

(2) 参数说明：`Integer` 类型的 ID;

(3) 调用函数说明：

检查参数，对入参进行判空处理，参数为空则记录错误信息，直接结束；

查询对象，参数不为空则调用 `UserService` 的 `getByID` 方法，`UserService` 调用 `UserDao` 的对应接口查询用户信息，查询不到则提示用户“此用户不存在”等相关信息，直接结束；查询到则返回 `User` 对象数据，根据界面需求显示。

2) 用户信息修改功能

(1) 方法名：`updateUser`;

(2) 参数说明：分为三类，一类是 `Integer` 类型的用户 ID，`String` 类型的昵称，`String` 类型的描述，`String` 类型的邮箱；一类是 `Integer` 类型的用户 ID，`byte[]` 类型的头像；一类是 `Integer` 类型的用户 ID，`String` 类型的老密码，`String` 类型的新密码，`String` 类型的确认密码；

(3) 调用方法说明：

检查参数，对各个参数进行有效性检查，无效则输出相应错误信息，立即结束；有效则调用 `UserService` 的 `getByID` 方法，`UserService` 调用 `UserDao` 的对应接口查询用户信息，查询不到则提示用户“此用户不存在”等相关信息，直接结束；

修改对象，查询到则返回 User 对象数据，调用 User 对象的 set 方法设置新数据到对象中。

存储对象，调用 UserService 的 update 方法，UserService 调用 UserDao 的对应接口更新相应的用户信息。

系统个人信息设置界面如图 4.1 所示。



图 4.1 系统个人信息设置界面

4.2.3 博文管理模块实现

博文管理模块主要涉及到的功能有博文浏览、博文编辑、博文搜索以及博文的管理，主要包括博文的增删改查操作，主要涉及对象 Blog、BlogController 以及主要方法 create、delete、update、get。这四大操作本身的业务逻辑并不复杂，主要对应数据库的 insert、delete、update、select 四大操作，根据参数执行对应方法获得数据。

1) 博文编辑功能

博文编辑包括博文的创建和修改操作，修改时在创建的基础上查询博文对象是否存在，不存在则提示用户“博文 XXX 不存在”等相关信息，存在才将新数据设置到旧对象中，并更新数据库，这里主要说明创建方法。

(1) 方法名：createBlog;

(2) 参数说明: Blog 对象, 包含必穿参数 String 类型的标题, 限制长度; String 类型的内容; Boolean 类型的是否公开的博客状态; Boolean 类型的是否允许评论的博客状态; Date 类型的创建时间; String 类型的标签 (可选);

(3) 调用方法说明:

检查参数, 调用 `checkParams` 方法检查入参, 参数无效则对应提示用户“无效参数 XXX”等相关信息, 立即结束; 参数有效则进入后续操作。

补充设置信息, 通过调用 `generateID` 方法自动生成博客 ID, 调用 `ShiroUtils` 的 `getUserID` 方法获得用户 ID, 调用 `LabelController` 的 `getByName` 检查标签的存在情况 (存在则调用 `BlogService` 的 `addBlogLabel` 方法增加关联关系; 不存在则先调用 `LabelService` 的 `add` 方法创建标签, 然后再建立博客与标签之间的关系); 最后调用 `Blog` 对象的 `set` 方法将所有信息设置到新创建的 `Blog` 对象中。

存储对象, 调用 `BlogService` 的 `add` 方法, `BolgService` 调用 `BlogDao` 的对应接口执行博客对象的插入数据库操作。

博文编辑界面如图 4.2 所示。



图 4.2 博文编辑界面

2) 博文浏览功能

博文浏览主要涉及到的是博文的查询操作, 查询操作有按 ID 查询、按名字模糊

查询、按分页参数进行集合查询。按 ID 查询可以为查看某一篇博文的内容，因其与互动管理模块紧密相连，所以这一功能将再互动管理模块进行说明。按名字模糊查询可以为在搜索框查询相应博文，因为搜索框设置的是对博文和标签的同时查询，所以这一部分在标签管理模块中说明。按分页参数条件查询可以为系统的博文展示，也可以为标签的分类展示。

此功能中主要说明按分页参数进行集合查询，主要是按创建的时间逆序展示，即最新的在最前面。

(1) 方法名: `getBlogs`;

(2) 参数说明: `int` 类型的 `limit`, 设置每页的博文数量; `int` 类型的 `offset`; 设置页数;

(3) 方法调用说明:

检查参数, 进行参数的判空以及范围检查, 无效输出错误信息, 直接结束; 有效继续后续操作。

获取数据, 调用 `BlogService` 的 `list` 方法, `BlogService` 调用 `BlogDao` 的对应接口, 通过以时间排序, 获取最新的博文集合, 再根据页面需求以及页面的参数进行展示。

系统博文浏览界面如图 4.3 所示。

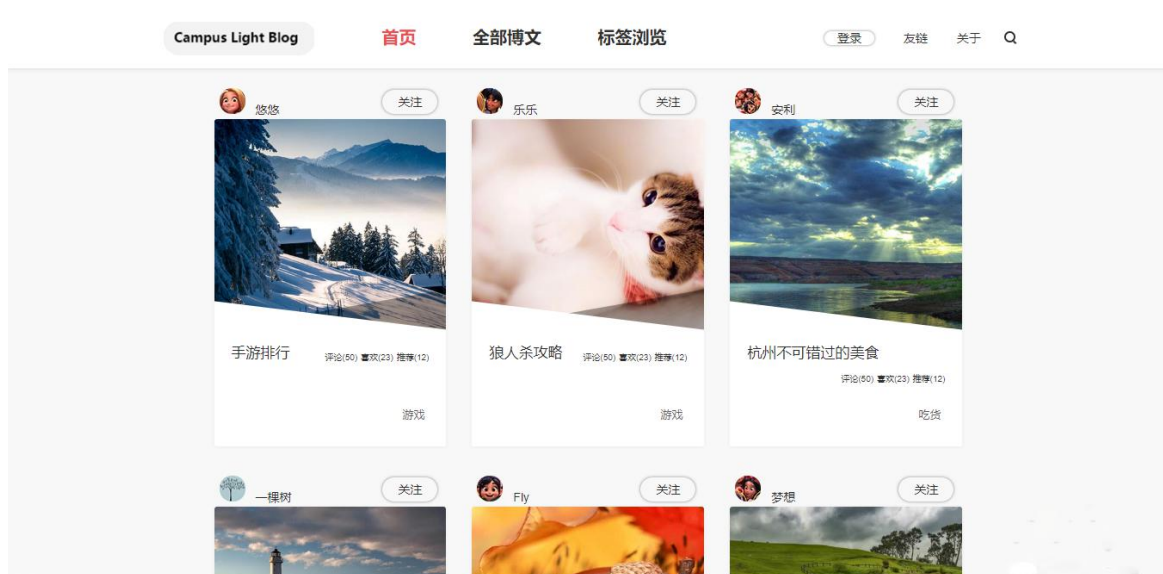


图 4.3 系统博文浏览界面

3) 博文管理功能

博文管理是后台管理主要功能之一，管理员具有最高权限，对所有博文都具有删改查的权限，也可以作为普通用户，发表博文。在此功能中主要说明删除操作，删除操作从参数个数上来看有单个删除 `remove` 和批量删除 `batchRemove` 两种操作；从参数类型上来看有 `deleteByID` 根据博文 ID 删除博文，`deleteByUserID` 根据用户 ID 批量删除博文，`deleteByLabelID` 根据标签 ID 批量删除博文。这里的操作主要依赖于多样化的查询，核心功能还是 `deleteByID`，所以这里主要说明 `deleteByID` 方法。

(1) 方法名：`removeBlog`;

(2) 参数说明：`Integer` 类型的博客 ID;

(3) 调用方法说明：

检查参数，对参数进行判空检查，为空则输出错误信息，直接结束；不为空则调用 `BlogService` 的 `getByID`，`BlogService` 调用 `BlogDao` 的对应接口查询博客是否存在数据库中，不存在则提示用户“博客 XXX 不存在”等相关信息；存在则继续后续操作。

删除对象，调用 `BlogService` 的 `deleteBlogRelations` 方法，`BlogService` 调用 `BlogDao` 的对应接口删除与博客相关联的对象的对应关系，如标签、评论等等，最后调用 `BlogService` 的 `deleteByID` 方法，`BlogService` 调用 `BlogDao` 的对应接口从数据库中删除博文。

后台管理之博文管理界面如图 4.4 所示。

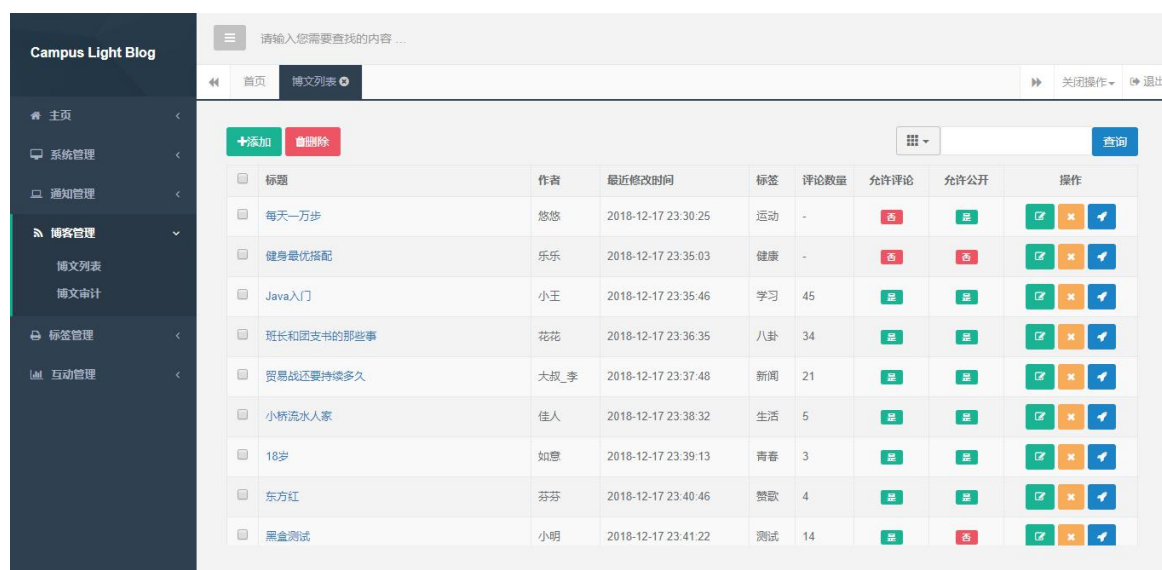


图 4.4 后台管理之博文管理界面

4.2.4 标签管理模块实现

标签管理模块主要是针对标签的增删改查操作，标签的操作与博文密切相关，同时因为博文和标签的关系是多对多的，所以在涉及到标签的相关操作时，需要考虑到博文标签关系表的相关操作。这里主要说明标签的创建过程以及标签的查询过程。

1) 标签创建功能

作为普通用户，当编辑博文时，在添加标签输入框中输入未创建过的标签，即可促发标签的创建操作；作为管理员，可直接创建标签。这里主要说明随博文的创建产生标签。

方法名：createLabel;

参数说明：Label 对象，包括主要参数 String 类型的标签名字；

调用方法说明：

检查参数，对参数进行判空和长度限制的检查操作，无效则输出错误信息，直接结束；有效才继续添加操作。

存储对象，通过调用 LabelService 的 add 方法，LabelService 调用 LabelDao 的对应 insert 接口创建标签，插入数据库。

2) 标签查询功能

标签的查询分为按 ID 查询，按名字模糊查询，按分页参数条件查询。标签的修改和删除都需要通过按 ID 查询标签进行后续操作，标签的查询则需要按名字模糊查询进行后续操作，博文的按标签分类展示需要按分页参数条件查询。这里说明按名字模糊查询和按分页参数条件查询。

(1) 按名字模糊查询

此功能调用 BlogService 的 getListByName 方法，以及 LabelService 的 getListByName 方法，并对名字参数做“%名字%”包装，在博文和标签表中分别进行模糊查询，将最后得到的查询结果分别放入两个集合中，并以 blogs 和 labels 作为这两个集合的 key，以 json 格式返回给前端，前端根据具体显示需求展示结果。

搜索资源展示界面如图 4.5 所示。



图 4.5 搜索资源展示界面

(2) 按分页参数条件查询

此功能调用 `LabelService` 的 `list` 方法，`LabelService` 调用 `LabelDao` 的对应接口，通过以时间排序，获取最新的标签集合，再根据页面需求以及页面的参数进行展示。

标签浏览展示界面如图 4.6 所示。

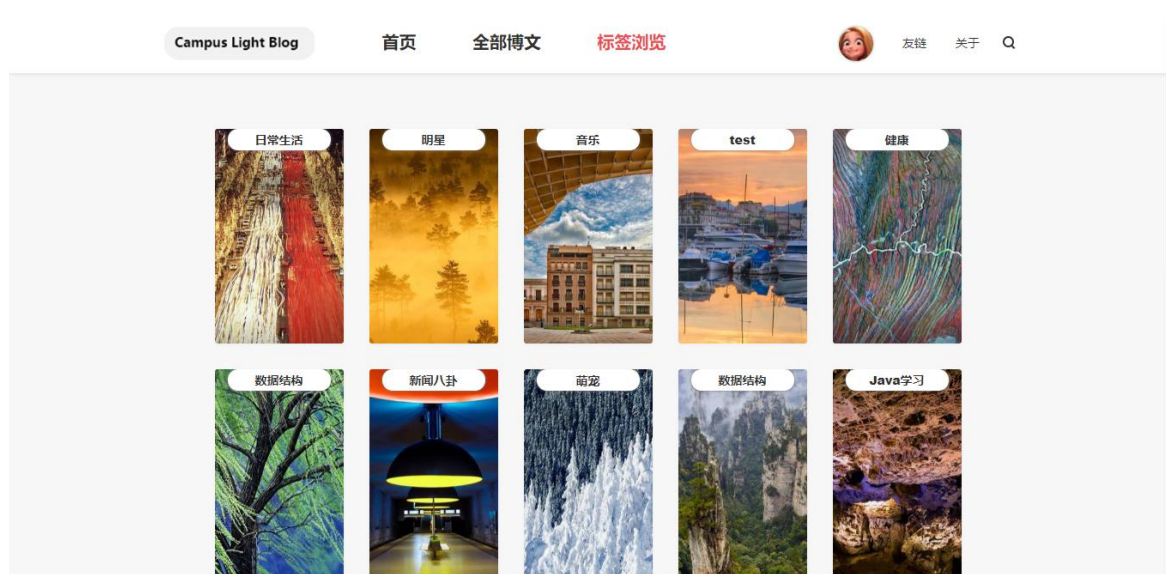


图 4.6 标签浏览展示界面

4.2.5 互动管理模块实现

互动管理模块包括评论管理、喜欢推荐管理、举报管理、黑名单管理，以及其他互动管理。主要涉及对象 User, Blog, Comment, ReplyComment, Like, Recommend, BlackUser 等等，以及 InteractionController，因为互动多为用户与用户之间的互动或者是用户与博文之间的互动（虽然本质上仍为用户与用户之间的互动），所以涉及的方法也相对比较多，这 5 个子管理模块都有各自的增删查操作，操作过程差不多，所以这里仅以评论的部分实现作为例子说明。

1) 评论和回复评论功能

评论的交互主要是在用户与博客之间的交互，用户给自己欣赏的博客进行评论，博主对评论进行回复，用户也可以对回复进行回复操作。根据数据库表的设计，将评论和回复设计为两张表，并且回复评论有两种类型，一种是直接对评论进行回复，一种是对回复进行回复，所以回复评论比评论的业务逻辑多了一些检查当前回复的是评论还是回复评论，以及设置回复评论的父回复评论（如果回复的是评论，则当前回复的父回复就是这个评论；如果回复的是回复，则当前回复的父回复就是这个回复）以及根回复评论的工作。这里涉及的方法主要就是 addComment 以及 addReplyComment，这里主要说明 addComment 方法。

（1）方法名：addComment；

（2）参数说明：Comment 对象，包括必要参数 Integer 类型的博文 ID，String 类型的评论内容；

（3）调用方法说明：

检查参数，调用 checkParams 方法对入参进行有效性判定，无效则输出错误信息，直接结束；有效则调用 BlogService 的 getByID 方法，BlogService 调用 BlogDao 的对应接口查询博客是否存在数据库中；

检查对象状态，调用 Blog 的 get 方法得到博客对象的是否可评论的状态，不可评论则提示用户“当前博文 XXX 不允许评论”等相关信息；可评论则继续后续操作；

设置补充信息，调用 filterComment 方法过滤评论内容，补充设置评论 ID，当前发出评论操作的用户 ID，设置评论时间为当前系统时间，最后调用 Comment 对象的

set 方法将所有信息设置到新创建的 Comment 对象中；

存储对象，调用 CommentService 的 add 方法，CommentService 调用 CommentDao 的对应接口将评论内容存入数据库中；

后续操作，调用 NoticeController 的 addNotice 方法，创建相应通知。

2) 评论查看功能

评论查看一般除了查看当前评论，其评论的所有回复也需要显示出来。查看评论可以只查看一条评论及其回复，也可以查看所有评论及其回复，这里主要说明查看其中一条。

(1) 方法名：getOneComment；

(2) 参数说明：Integer 的评论 ID；

(3) 调用方法说明：

检查参数，对入参进行判空操作，参数为空则输出错误信息，直接结束；不为空则继续查询操作。

获取对象，调用 CommentService 的 getByID 方法，CommentService 调用 CommentDao 的对应接口查询评论是否存在数据库中；调用 getRepliesByCID 方法查询到评论下的所有回复。

其他人与博文进行互动的界面如图 4.7 所示。



图 4.7 部分互动界面

博文评论界面如图 4.8 所示。



图 4.8 博文评论界面

4.2.6 系统管理模块实现

系统管理模块包括用户注册、登录、找回密码和注销功能，以及用户的管理，主要包括用户的增删改查操作，主要涉及到用户对象 `User`、`SystemController`，和几个主要的方法：`register`、`login` 以及 `logout` 和 `cancleAccount`。

1) 用户注册功能

普通用户执行用户注册使用 `register` 方法，管理员的身份由已存在的管理员授权，借助用户 `id` 修改用户权限即可。这里主要说明 `register` 方法。

(1) 方法名：`register`;

(2) 参数说明：`User` 对象，包括必传参数值 `String` 类型的昵称，限制重名；`String` 类型的密码，但限制字符类型只能为数字字母或下划线，长度限制为 6-10；`String` 类型的描述，限制在 255 个字符以内；`byte[]` 类型的头像，限制长宽；`String` 类型的邮件，满足现有的邮件格式要求；

(3) 方法调用说明：

检查参数，调用 `checkParams` 方法检查所有输入参数的合法性，检查不通过则输出相应错误信息，直接结束；通过则调用 `UserService` 的 `getByName` 或 `getEmail`

方法，UserService 再调用 UserDao 对应方法对比数据库的数据，检查用户是否已经注册，已注册则提示用户“此邮箱已被注册，若需重新注册，请联系管理员”等相关信息，直接结束；没有注册才进行后续的注册工作。

首先调用 setInfos 方法为用户设置用户的唯一 ID，加密后的密码，用户角色（默认设置为 1，0 为管理员），用户注册时间（设置为当前系统时间）等必要信息；然后调用 User 对象的 set 方法将所有信息设置到新创建的 User 对象中；最后调用 UserService 的 add 方法，UserService 再调用 UserDao 对应方法保存用户信息到数据库。

用户注册界面如图 4.9 所示。

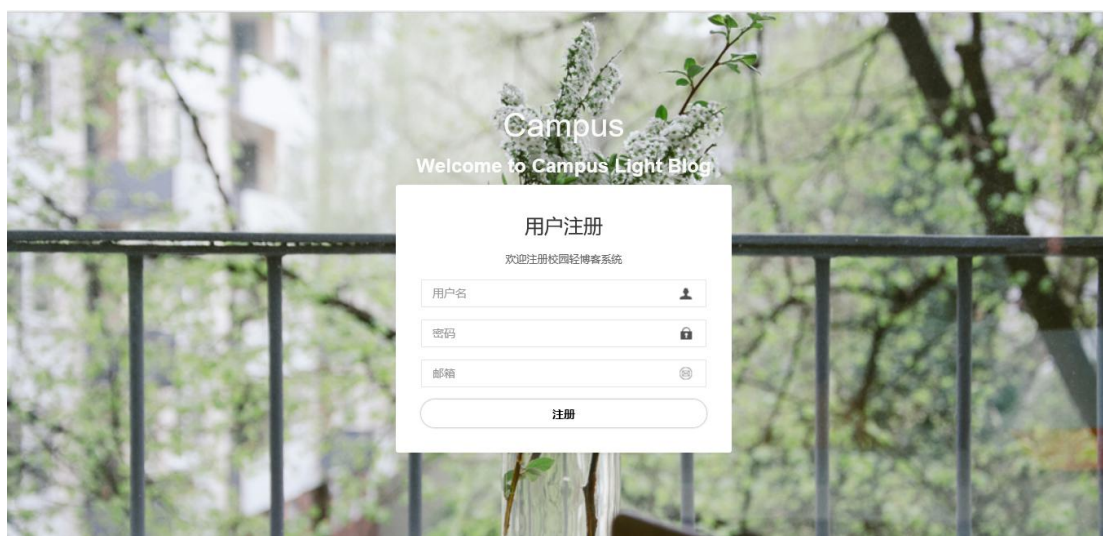


图 4.9 用户注册界面

2) 用户登录功能

用户通过输入用户名和密码进行登录检查。本系统采用 shiro 的安全性检查，通过调用 Subject.login 方法执行安全登录，这一部分将在系统安全实现部分详细介绍。用户退出可直接调用 shiro 的 logout 方法来执行，所以这里主要介绍 login 方法。

(1) 方法名：login；

(2) 参数说明：String 类型的用户名，以及 String 类型的密码；

(3) 方法调用说明：

检查参数，调用 Java 语言自带的 isEmpty 方法和比较 null 的方式对参数进行判

空检查，出现空值则输出相应错误信息，直接结束；均不为空则进行以下登录操作。

调用 MD5Utils 的 encrypt 方法为密码加密（存在数据库中的密码是加密后的密码），并将用户名和加密后的密码打包成 UsernamePasswordToken 类，然后调用 shiro 的 Subject 的 login 方法执行登录验证。

登录验证的大致过程为：调用 UserService 的 getByName 方法，UserService 调用 UserDao 的对应方法从数据库获得此用户信息，若用户不存在，则提示用户“此用户名不存在，请检查用户名”等相关信息，直接结束；根据得到的用户信息得到存储的密码，与加密后的输入密码进行比较判断，不同则提示用户“密码错误，请检查密码”等相关信息，直接结束；相同则返回成功登录的相关信息，跳转至主页面。此方法将在 4.3.1 身份验证以及授权小节中详细说明。

系统登录界面如图 4.10 所示。

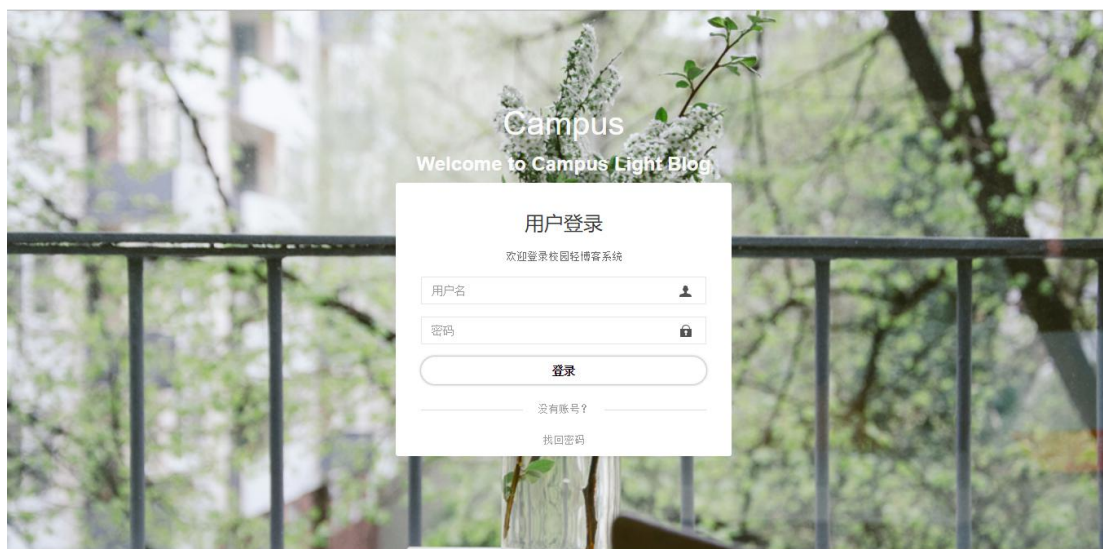


图 4.10 系统登录界面

3) 用户管理功能

系统后台管理中主要功能之一即是用户管理功能，管理员作为最高权限者，此功能模块包含的是用户的增删改查操作，这里的操作主要是针对用户对象的，这里以创建用户功能说明。

管理员创建用户与普通用户注册的操作没有很大的区别，其区别在输入参数中除

了基本的用户信息外，还有 short 类型的角色类型，以及 short 类型的账号状态类型，所以可通过调用 UserService 的 add 方法，UserService 再调用 UserDao 对应方法创建管理员类型的用户以及账户状态不可用的用户。

后台管理之用户管理界面如图 4.11 所示。

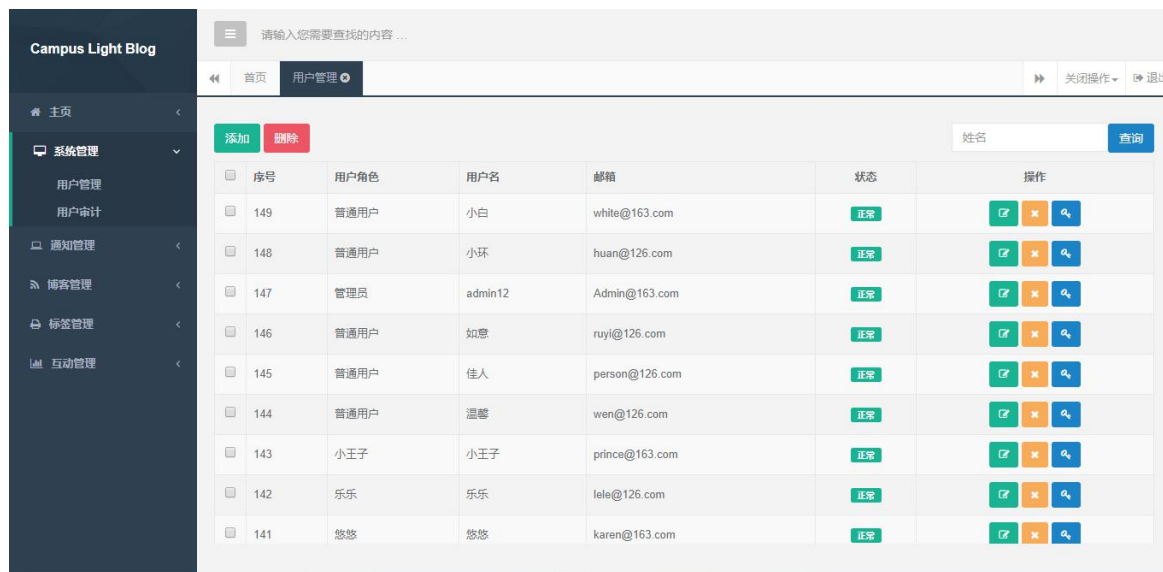


图 4.11 后台管理之用户管理界面

4.3 系统安全实现

在 3.5 系统的安全设计中提到系统的分层结构、多层验证、安全编码、MD5 盐值加密和用户权限控制等五个方面，这五个方面中分层结构与架构设计有直接关联，这一点在系统公共部分说明中已经详细阐述，而多层验证分布在前端模板和后端访问方法接口的验证代码中，不单独进行说明。安全编码中涉及到的防止脚本注入、防止 SQL 注入等防御功能，这里将主要描述 xss 防止跨站脚本攻击的安全防御功能实现。用户信息加密和用户访问控制部分与 Apache Shiro 的安全管理相关，这个部分将主要描述用户的登录验证和用户权限验证的 shiro 实现。

4.3.1 身份验证以及授权

在使用 shiro 的相关操作前需要对 shiro 进行相关配置，本系统采用注解的方式，通过创建 ShiroConfig 类来设置相关信息，部分属性值存储在 application.yml 中，如

redis 的相关配置等。

ShiroConfig 类中需配置以下相关信息，并借助@Bean 注解进行管理使用：

(1) getLifecycleBeanPostProcessor(), 静态方法, 获得 LifecycleBeanPostProcessor 类实例以控制 shiro bean 对象的生命周期;

(2) shiroDialect(), 获得 ShiroDialect 类实例, 方便在模板中使用 shiro 标签; shiroFilterFactoryBean(SecurityManager securityManager), 获得 ShiroFilterFactoryBean 类实例, 配置过滤内容, 通过 shiroFilterFactoryBean 的 setLoginUrl("/login") 方法配置登录 URL, setSuccessUrl("/index") 方法配置成功登录 URL, setFilterChainDefinitionMap 方法设置拦截规则, 其中规则常用的有这三种: "/login", "anon" 表示可以匿名访问, "/logout", "logout" 表示通过 shiro 自带的 logout 即可访问, "/*", "authc" 表示需通过验证才可以访问;

(3) userRealm(), 获得 UserRealm 类实例, UserRealm 类继承于 Realm 类, shiro 的 Realm 类主要负责设置用户验证和用户授权的具体操作, 为了配合系统需求, 需要自定义具体的验证和授权操作, 所以这里实质上是 UserRealm 类的设置过程;

(4) redisCacheManager(), 获得 RedisCacheManager 类实例, 配置 redisManager 类以实现借助 redis 进行缓存管理;

(5) sessionManager(), 获得 DefaultWebSessionManager 类实例, 配置相关参数以进行 session 管理;

(6) securityManager(), 获得 SecurityManager 类实例, 配置 realm, cacheManager, sessionManager 等参数以进行安全管理。

通过 shiro 的配置构建了 SecurityManager 环境, 接着借助 shiro 的登录验证的功能实现系统的用户登录验证; 借助 shiro 的权限检查功能实现系统的用户授权访问。这两大功能与 Subject, SecurityManager, Authenticator, Authorizer, Realm 等类有着紧密的联系。

(1) 用户登录验证

为定义用户登录验证的规则, 系统定义了 UserRealm 类, 此类继承于 Realm, 通过重写 doGetAuthenticationInfo(AuthenticationToken var1) 方法自定义登录验证规则。

用户登录验证的整体实现过程为：对提交表单进行判空，长度判断等基本检查后，调用 MD5Utils 的 encrypt 方法对用户密码进行 2 次 MD5 盐加密后得到加密后的密码，调用 UsernamePasswordToken 的构造函数将用户名和加密后的密码包装成相应的类，然后以包装后的类作为参数，调用 Subject 的 login 方法，之后 Subject 调用 SecurityManager 的 authenticate 方法，SecurityManager 调用 Authenticator 的 authenticate 方法，Authenticator 调用 Realm 的 doGetAuthenticationInfo 方法，最后由 UserRealm 重写的对应方法执行自定义的登录验证。

自定义的登录验证实现为调用 AuthenticationToken 的 getPrincipal 方法和 getCredentials 方法分别获取用户名和加密后的密码，UsernamePasswordToken 继承于 AuthenticationToken，调用 UserService 的 getByName 获取用户信息，并检查用户状态是否可用，与数据库存储的密码进行比较。所有操作出错则抛出相应异常，立即结束；操作均正确则通过 SimpleAuthenticationInfo 的构造函数，以及参数用户对象，密码和当前 realm 的名字等构造验证 AuthenticationInfo 验证信息对象，将其存在 subject 中，返回正确消息，并跳转至系统主页面。

（2）用户授权访问

为定义用户权限信息获取的规则，系统在 UserRealm 类中通过重写 doGetAuthorizationInfo(PrincipalCollection var1)方法自定义权限获取方式。

用户授权访问的整体实现过程为：用户访问的控制在后端代码中借助的是注解 @RequiresPermissions 来检查用户权限，所以需要利用 Spring 的 AOP 面向切面编程的特性，借助 shiro 配置中的 AuthorizationAttributeSourceAdvisor 通知类，来拦截并检查用户是否具有相关权限。

自定义的权限获取方式的实现过程为：系统通过调用 ShiroUtils 的 getUserId 方法获取用户 ID，并通过 UserService 的 getById 获得用户，从而获得用户角色，然后通过调用 SystemService 的 getAuthInfosByRole 方法获取用户权限字符串集合，最后通过调用 SimpleAuthorizationInfo 的 set 方法设置字符串集合格式的用户权限信息，并返回权限信息。

用户在访问某个需要检查权限的方法时，系统拦截方法，转去获取方法上注解

所具有的权限，然后通过调用 Subject 的 checkPermission 方法，Subject 调用 SecurityManager 的 checkPermission 方法，SecurityManager 调用 Authorizer 的 checkPermission 方法，Authorizer 调用 Realm 的 doGetAuthorizationInfo 方法，最后由 UserRealm 重写的对应方法来自定义权限获取方式，以获取当前用户所具备的全部权限，比较用户访问方法需要的权限与用户所具备的所有权限，如果包含则可以继续访问，不包含则抛错，返回上一操作界面。

4.3.2 xss 内容过滤

为防止常见的 xss 跨站脚本攻击，系统设置了 xss 过滤器对用户提交数据，如博文或评论，进行检查和过滤，并借助 Jsoup 工具的 HTML 文档清理功能来做进一步的过滤工作。

在使用 Filter 之前需要进行相关配置，系统采用注解，非 xml 配置的方式创建了 XssConfig 类作为配置类，主要需要配置 filter 的注册类 FilterRegistration- Bean，设置 filter 的配置参数，如 filter 实例，初始化参数，filter 拦截的 url 地址等等。

系统通过定义 XssFilter 以及 XssHttpServletRequestWrapper，以及借助 Jsoup 工具定义一些过滤接口，存放在 JsoupUtil 类中，这里主要介绍 XssFilter 类和 XssHttpServletRequestWrapper 类的内容。

(1) XssFilter 类

XssFilter 继承了 Servlet 的 Filter 类，可以对 request 和 response 进行过滤，XssFilter 重写了 doFilter(), init(), destroy() 方法，init() 方法主要初始化一些配置中的预置参数，比如预置过滤内容，destroy() 方法是在 filter 工作结束后处理的工作，doFilter() 方法则是核心的过滤的操作，所以这里主要说明 doFilter() 方法。

doFilter() 的实现过程为：首先对参数进行相应类型的转换，由 ServletRequest，ServletResponse 转换为 HttpServletRequest，HttpServletResponse，调用 handleExcludeURL 方法检查 url 中是否存在于 xssConfig 预置白名单 url（包括静态资源的 url 等）中，如果在，则不做 xss 的过滤，由 filterChain 接着找到其他的 filter 做 doFilter 方法；若不在，则将 request 转换为自定义的 HttpServletRequestWrapper 类，重新定义 request 获取参数的操作，再利用 Jsoup 定义具体的 xss 过滤规则对 request

的参数名和参数值进行相应过滤。

(2) XssHttpServletRequestWrapper 类

XssHttpServletRequestWrapper 类继承了 HttpServletRequestWrapper 类，实现了 HttpServletRequest 接口，重写了 getParameter, getParameterValues, getHeader 等方法。这三个方法都是先过滤入参，后取原始数据，然后按照自定义的过滤方法对原始数据进行过滤，返回过滤后的数据。这里我就仅针对 getParameter 方法进行说明。

getParameter 方法的实现过程为：首先调用 JsoupUtil 的 clean 方法过滤参数名，然后调用 super.getParameter(String name)方法获得参数名对应的原始参数值，最后调用 JsoupUtil 的 clean 方法过滤参数值，并返回过滤后的参数值。

4.4 系统测试

系统测试是指基于一些规定条件来操作程序以检测程序的错误，测量软件质量并评估其是否满足设计要求的过程。系统测试的目的是为了找到系统运行过程中不符合系统需求的地方。

系统测试主要通过大量的测试用例对系统的功能点进行逐个检测，大致通过三个方面去检测系统是否达到预期的要求：检测该实现的功能是否及时执行，该提示的错误信息是否正常显示，该应对的安全防御是否正确执行。

4.4.1 测试方法

系统主要基于白盒测试和黑盒测试进行测试。白盒测试，又称透明盒测试，结构测试等，是基于程序内部结构的测试，主要通过检查代码在条件，路径，溢出等句法上的错误来指出结构上的缺陷。黑盒测试，又称功能测试，数据驱动测试等，是基于系统外在功能的测试，主要通过有效输入和无效输入来验证系统功能，以及外部结构是否正确反馈。

白盒测试与系统代码息息相关，所以随开发过程进行，并借助 Junit、findbugs 等工具进行，这里主要介绍与系统功能有紧密关系的黑盒测试。

4.4.2 测试用例

测试用例分为三大类：系统功能测试用例、系统性能测试用例和系统安全测试用例。测试用例本应覆盖系统的所有功能、所有异常状况、所有安全检查，但因篇幅限制，将抽选部分典型用例进行说明。

(1) 系统功能测试用例，功能反馈包括有效输入的正常反馈测试用例，即系统对于正确输入应具有响应操作；无效输入的系统功能异常反馈测试用例，即系统对于错误输入应具有提示操作。

正常反馈测试用例的部分用例具体描述如表 4.1 所示。

表 4.1 正常反馈测试用例表

编号	测试项	前提条件	操作	输入	预期结果
U001	用户注册	用户已通过在浏览器输入系统服务器地址访问系统的主页面	1. 点击主页上的“用户注册”按钮 2. 填写注册信息 3. 点击注册页面上的“提交”按钮	正确填写注册信息 昵称: test 密码: 123 描述: test 邮箱: 123456@qq.com	1. 跳转到“注册”页面 2. 提示“注册成功” 3. 跳转到登录后的主页面
U002	用户登录	用户已通过在浏览器输入系统服务器地址访问系统的主页面	1. 点击主页上的“用户登录”按钮 2. 填写登录信息 3. 点击登录页面上的“登录”	正确填写登录信息 用户名: test 密码: 123	1. 跳转到“登录”页面 2. 提示“登录成功” 3. 跳转到登录后的主页面
B001	发布博客	用户有效且用户已正确登录	1. 点击“写博文”按钮 2. 正确填写博文信息 3. 点击“发布”按钮	填写合法博文信息 标题: title 内容: test	1. 跳转到“博文编辑”页面 2. 提示“发布”成功 3. 可在“我的博文”中查看新发布的文章
B002	修改博客	用户有效且用户已正确登录, 并进入“我的主页”中, 页面上显示所有已发表的博客	1. 点击要修改的文章的“编辑”按钮 2. 正确填写修改内容 3. 点击“保存”按钮	填写合法博文修改信息 标题: title 内容: testtest	1. 跳转到“博文编辑”页面(已包含旧的数据) 2. 提示“保存”成功 3. 可在“我的博文”中查看修改后的文章

华中科技大学硕士学位论文

续表 4.1 正常反馈测试用例表

B003	删除博客	用户有效且用户已正确登录，并进入“我的主页”中，页面上显示所有已发表的博客	1. 点击要删除的文章的“删除”按钮 2. 阅读删除前的提示信息 3. 点击“确认删除”按钮		1. 弹出删除提示框 2. 刷新“我的主页”，删除的文章不再出现
B004	查询博客	用户至少正在访问系统的主页面	1. 点击搜索框 2. 正确输入查找信息	正确输入查询信息 查询: test	1. 跳转到查询结果界面，显示与“test”相关的用户，博文以及标签等信息
I001	发表评论	用户有效且用户已正确登录，且评论的博文存在	1. 点击要评论的文章的“评论”按钮 2. 输入评论信息 3. 点击评论框旁的“确认”按钮	输入合法评论信息	1. 弹出评论窗口，并显示现有评论信息 2. 提示“评论成功” 3. 刷新当前评论窗口，显示刚才新加评论
I002	删除评论	用户有效且用户已正确登录，并处于“我的主页”中，页面上显示所有已发表的博客，且评论的博文存在	1. 点击需要删除的博文评论旁的“删除”按钮 2. 阅读删除前的提示信息 3. 点击“确认删除”按钮		1. 弹出删除提示框 2. 刷新“我的主页”，删除的文章评论不再出现

异常反馈测试用例的部分用例具体描述如表 4.2 所示。

表 4.2 异常反馈测试用例表

编号	测试项	前提条件	操作	输入	预期结果
U003	用户异常注册	用户进入注册界面，除昵称外，其他参数均有效	输入含非法字符的昵称	test&*	昵称输入框旁弹出“昵称仅支持字符，数字，以及下划线，且不超过 64 个字符”提示信息
			输入超过 64 字符的昵称	Test...123	
			输入已存在的昵称	佳人	昵称输入框旁弹出“昵称已存在”提示信息
		用户进入注册界面，除密码外，其他参数均有效	输入长度少于 6 个字符的非法密码	123	密码输入框旁弹出“密码长度应在 6-10 个字符之间”提示信息
			输入长度多于 10 个字符	Testtesttest	

续表 4.2 异常反馈测试用例表

U003	用户异常注册	用户进入注册界面, 除描述外, 其他参数均有效	输入超过 255 个字符的描述	啦啦啦啦...啦	描述输入框旁弹出“描述长度应不大于 255 个字符”提示信息
		用户进入注册界面, 除邮箱外, 其他参数均有效	输入非法邮箱	123qq.com	邮箱输入框旁弹出“邮箱格式不正确”提示信息
U004	用户异常登录	用户进入登录界面, 除用户名外, 其他参数均有效	不输入用户名		用户名输入框旁弹出“用户名不能为空”提示信息
			输入未注册过的用户名	Kaka	界面弹出“用户名或密码错误”提示信息
		用户进入登录界面, 除密码外, 其他参数均有效	不输入密码		密码输入框旁弹出“密码不能为空”提示信息
			输入错误密码		界面弹出“用户名或密码错误”提示信息
B005	异常发布博客	用户已正确登录, 并进入博客编辑界面, 除标题外, 其他参数均有效	不输入标题		标题输入框旁弹出“标题不为空”提示信息
			输入长度超过 64 个字符的标题	The biggest supri...we	标题输入框旁弹出“标题长度应不大于 64 个字符”提示信息
		用户已正确登录, 并进入博客编辑界面, 除内容外, 其他参数均有效	不输入内容		内容输入框上弹出“内容不为空”提示信息
I003	异常发表评论	用户已正确登录, 并进入博客浏览界面	不输入评论内容		评论输入框上弹出“评论内容不为空”提示信息
			输入超过 255 个字符的内容	I am a girl ... bye	评论输入框上弹出“评论内容长度应不大于 255 个字符”提示信息

(2) 系统性能测试用例

以系统性能和可靠性的部分用例进行说明, 具体描述如表 4.3 所示。

华中科技大学硕士学位论文

表 4.3 性能测试用例表

编号	测试项	测试操作说明	预期结果
P001	性能	登录系统，记录页面跳转响应时间	平均响应时间不超过 2 秒
		编辑博文，记录提交操作响应时间	平均响应时间不超过 2 秒
		搜索资源，记录反馈结果响应时间	平均响应时间不超过 2 秒
		8000 用户同时执行登录操作	平均响应时间不超过 10 秒
P002	可靠性	以 24 小时作为单位，单人操作，记录系统可用时间	可用时间与总测试时间之比超过 99%
		以 24 小时作为单位，多人同时操作，记录系统可用时间	可用时间与总测试时间之比超过 99%

(3) 系统安全测试用例

以界面拦截和内容过滤的部分用例进行说明，具体描述如表 4.4 所示。

表 4.4 安全测试用例表

编号	测试项	异常说明	操作	输入	预期结果
S001	界面拦截	直接通过 url“暴力”访问需要用户权限的地址	用户未登录，通过 url 暴力访问博文编辑界面	https://192.168.0.21/blogs/add	跳转到“登录”界面
			用户登录，通过 url 暴力访问后台管理界面	https://192.168.0.21/blogs/admin	跳转到“登录”界面
S002	内容过滤	用户提交一些脚本，非法标签等可能影响后台操作的内容	用户在 url 上加入指定标签	https://192.168.0.21/blogs/search?content="<script>x=document.cookie;alert(x);</script>"	url 过滤后访问，显示“没有相关结果”，跳转至首页
			用户在博文评论中提交标签	onclick="alert("test");return false;	后台将相应输入转换为字符串存储，无异常弹窗

4.4.3 测试结论

系统根据测试用例进行了为期 3 天的测试，大部分测试用例均已通过，少数用例出现了问题，如提交博客提示成功后界面上还处于提交状态，少数内嵌页面的方法路径可直接通过 url 访问等等。根据测试结果经过了相应修正后，解决了大部分问题，能够保证系统的基本运行。

4.5 本章小结

本章阐述了校园轻博客系统的详细实现和测试，首先简单介绍了系统的开发环境，开发工具；然后通过系统代码结构，系统函数的调用情况，以及借助伪代码的方式代替具体代码实现等方式对系统的各个功能模块的具体实现进行了详细的介绍，并展示了部分功能显示界面；接着，为了实现系统的安全性，采用 apache shiro 的安全机制，介绍了 shiro 验证和授权在代码中的具体实现，并且为了防止常见的跨站脚本攻击，系统还集成了 xss 过滤的功能；最后根据系统功能、性能和安全性制定相应的测试用例，并借助黑盒测试对系统进行了测试。

5 总结与展望

5.1 全文总结

本文围绕主题——基于 Spring Boot 的校园轻博客系统的设计与实现，进行了三大核心问题的学习与研究：“是什么”，“为什么”，以及“怎么做”。这三个问题贯穿始终：从整体来说，论文的大体结构则是系统的核心内容是什么，为什么要设计这样的系统，以及系统应该怎么做；从部分来说，以需求分析举例，需求分析是什么，为什么要做需求分析，以及需求分析应该怎么做；从某一个点来说，以 Apache Shiro 技术为例：Apache Shiro 是什么，为什么要用 Apache Shiro，怎么通过 Apache Shiro 来达到最终目的？

本文在绪论章节中通过校园轻博客系统的研究背景与意义，国内外研究情况等大致回答了系统是什么，为什么要设计此系统问题；在关键技术研究章节中通过介绍技术的发展与运用回答了系统将使用的技术是什么，为什么采用这种技术去实现问题；而系统需求分析和设计以及系统实现和测试章节重点描述了校园轻博客系统的“怎么做”过程，也是系统开发过程的几大重要步骤：

（1）对系统的需求进行分析，通过对功能，用户角色以及用户角色对应的用例的分析，同时借助工具 Visio 绘制用例图，进一步清晰系统功能；并且通过对系统的性能、易用性、可靠性等除功能外的额外特性的分析完成了系统的非功能性需求分析，使得需求分析更加全面；

（2）根据系统的需求分析详细设计了系统的整体结构、数据库、以及安全性：对系统的体系结构，以及功能结构进行设计，得到系统的五大功能模块：用户信息管理模块、博文管理模块、标签管理模块，以及互动管理模块、系统管理模块，并借助功能结构以及功能流程图等表示方式对这五大模块的功能模块设计进行了详细说明；根据系统的功能结构进行相关数据库设计，并借助工具 PowerDesigner 设计相应的逻辑模型和物理模型，并借助生成的可执行的 SQL 文件在数据库中生成相应的数据库表；从系统结构的分层设计、前后端的多重验证、服务器端的安全编码、用

户密码 MD5 盐值加密以及用户权限控制的防止“暴力”访问等多个方面对系统进行了安全性设计，并对部分核心安全设计进行了具体说明；

(3) 确定系统开发环境，进而对系统的主要模块功能实现以及安全性实现进行描述。通过 IDEA 管理项目代码，借助 Spring Boot+MyBatis 框架实现系统基本功能，本文通过对功能核心函数的参数说明和调用函数说明来描述功能实现过程；借助框架特性以及代码结构特性保证系统性能；借助 Apache Shiro 框架实现系统安全性保障，本文对用户登录的身份验证、用户访问的授权控制以及提交内容的 xss 过滤三方面的实现进行了主要说明；

(4) 采用黑盒测试和白盒测试作为测试的主要方法，白盒测试随开发流程执行，测试阶段以黑盒测试为主。测试用例以三大类进行：功能测试、性能测试和安全测试。黑盒测试根据测试用例进行测试，记录测试结果，以便修正系统 bug。

5.2 展望

由于研究和实现的时间限制，以及一些外在和内在条件的限制，虽然校园轻博客系统已经初步实现，但是仍然还有需要补充完善，优化和扩展的地方，还需要进一步的思考和研究：

(1) 系统可以进一步集成博文推荐模块，根据用户的浏览记录等信息生成用户个性化信息，再根据其信息进行个性化推荐等等；

(2) 系统的后台管理部分目前对审计部份没有形成一个单独的模块，而是与系统的删除功能紧密相关，后续可以将审计部分单独化，形成相关日志，方便查询和统计；

(3) 系统在安全防护方面除了现有的跨站脚本攻击的过滤内容防御，还应针对其他常见攻击，如 DOS/DDOS 攻击等准备应对措施。同时需要部署防火墙等防御措施，以加固系统安全。

致 谢

论文写到这里，我才突然意识到不仅是我的论文，我的校园时光也即将结束，要感谢的事很多，要感谢的人也很多，要感谢的话更是说也说不完。

首先要感谢我的导师——卢力老师，不仅作为师者，为我们传道授业解惑；还作为临时父母，照顾着我们的身心健康；甚至作为朋友，为我们分忧解难。同时感谢导师让我认识了我们实验室的兄弟姐妹们，是他们和您让实验室成了我学校里的另一个“家”。还要感谢导师即使是在学业的最后阶段还要为我的论文不厌其烦地提出修改意见，而且在我遇到问题或阻力时，不仅给我勇气去面对，也给我方向去探索。

然后要感谢培育我的华中科技大学，为学子们提供了优美的校园环境，海量的学习资源，优秀的师资资源，良好的学习氛围，丰富的学习形式，舒适的校园生活等等。

接着要感谢我的父母，是他们教会我认真负责、积极乐观，也是他们为我创造一片坚实后方，我才能在前方勇往无前。还要感谢我的朋友们，是他们让我看到一个或天真、或成熟、或开心、或悲伤、或糊涂、或清醒的小世界，感谢他们给我机会去参与他们的生活，同样感谢他们陪我走过这段路。

最后要感谢这方天地，感谢在这方天地里遇到或错过的所有人。

参考文献

- [1] Rudolf Ammann. Jorn barger, the newspaper network and the emergence of the weblog community. In: Proceedings of the 20th ACM conference on Hypertext and hypermedia, Italy: Torino, 2009: 279-287
- [2] 陈健美. 埃文·威廉姆斯: “喜新厌旧”的 Twitter 教父. 金融博览(财富), 2013(24): 83-85
- [3] Karen Latchana Kenney. David Karp: The MasterMind Behind Tumblr. US: Lerner Pub Group, 2013: 80-82
- [4] 罗攀. 博客自媒体发展面面观. 今媒体, 2009(2): 76-78
- [5] 韩净. 中国博客发展问题探讨. 北方经贸, 2011(5): 59-60
- [6] 谢耘耕, 徐颖. 微博的历史、现状与发展趋势. 现代传播, 2011(4): 75-80
- [7] 苏丽娟. 浅议轻博客. 内蒙古科技与经济, 2012(9): 85-86
- [8] 小刀. 轻博客悄然兴起. IT 时代周刊, 2011(19): 116-117
- [9] 杨晓新, 章伟民. 博客在教育中的应用研究. 中国远程教育, 2006(6): 49-52
- [10] 丹尼斯·麦奎尔, 斯文·温德尔. 大众传播模式论. 祝建华译. 上海: 上海译文出版社, 1987: 92
- [11] 刘文秀, 左明章, 张昆明. 博客在高校校园文化建设中的应用研究. 中国教育信息化, 2006(7): 47-48
- [12] 徐满, 曹艳. 利用高校教师博客 开展隐性专业教育. 边疆经济与文化, 2013(5): 118-119
- [13] 郑建芳. 让班级博客成为智慧摇篮. 华夏教师, 2017(9): 93
- [14] 王君丹. 学科博客在高校的应用与实现. 无线互联科技, 2018(3): 88-90
- [15] 王月. 轻博客的传播特点及对传播生态的影响. 中州学刊, 2014(2): 173-176
- [16] Trygve Reenskaug, James Coplien. The DCI Architecture: A New Vision of Object-Oriented Programming. Programmer, 2009(6): 99-103
- [17] F Buschmann , K Henney. Pattern-Oriented software architecture. US: John Wiley & Sons Inc, 2004: 41-42

- [18] 段升杰. 浅谈软件三层架构开发. 信息与电脑: 理论版, 2010(5): 44-45
- [19] 孙亚志. 浅谈 Java web 技术 struts2 框架的 MVC 模式. 数字技术与应用, 2013(5): 208-210
- [20] 张雪敏. 浅议 MVC 设计模式在 Java Web 中的作用. 科技风, 2018(22): 58-59
- [21] Clarence Ho, Harrop Rob, Schaefer Chris. Pro Spring. US: Springer, 2014: 12-32
- [22] Marten Deinum, Daniel Rubio, Josh Long. Spring 5 Recipes. US: Apress, 2017: 27-32
- [23] Renaud Pawlak, Lionel Seinturier, Jean-Philippe Retailé. Foundations of AOP for J2EE Development. US: Apress, 2005: 121-124
- [24] Rod Johnson. Expert One-on-One J2EE Design and Development. UK: Wrox Press Ltd, 2002: 45-46
- [25] Felipe Gutierrez. Spring Boot. US: Apress , 2017: 7-16
- [26] 吕宇琛. SpringBoot 框架在 web 应用开发中的探讨. 科技创新导报, 2018(8): 168-173
- [27] 张峰. 应用 SpringBoot 改变 web 应用开发模式. 科技创新与应用, 2017(23): 193-194
- [28] K. Siva Prasad Reddy. Beginning Spring Boot 2. US: Apress, 2017: 65-69
- [29] 荣艳冬. 关于 Mybatis 持久层框架的应用研究. 信息安全与技术, 2015(12): 86-88
- [30] 贺军, 焦荣. 基于 XML 应用的 MyBatis 技术. 火力与指挥控制, 2013(z1): 86-88
- [31] Liu Tiehang. Research of a Code Generator Based on Integration of SSH2 and Apache Shiro. e-Science Technology & Application, 2013, 4(4): 82-88
- [32] 马亚娜, 钱焕颜, 孙亚民. Cookie 在 web 认证中的应用研究. 小型微型计算机系统, 2004(2): 21-22
- [33] YANG Yun-ping, WU Zhi-jun. Application of Apache Shiro Security Framework in Technology Transfer Services System. Computer & Modernization, 2014(3): 158-160
- [34] 韩坤. Web 服务安全会话管理的研究与实现: [硕士学位论文]. 北京: 北京邮电大学图书馆, 2007

- [35] 李厚刚, 白玉芹. 加密算法在共享软件加密中的简单应用. 网络安全技术与应用, 2010(9): 12-13
- [36] 王保华, 薛锦云, 丁树良. Java 多线程机制及其应用. 计算机与现代化, 2000(1): 1-6
- [37] 丁洁. 基于 Shiro 的 Web 应用安全框架设计研究. 信息与电脑 (理论版), 2018(13): 38-39
- [38] 荣艳冬, 冯建平. 深入探索 Shiro 权限框架的体系结构和实际应用. 电脑知识与技术, 2015, 11(22): 67-68
- [39] 孟亚辉. 浅谈软件项目开发过程中的需求分析. 科技信息, 2009(11): 435-436
- [40] 陈琳. 网络信息内容审计技术研究: [硕士学位论文]. 南京: 南京航空航天大学图书馆, 2012
- [41] Andrew Stellman, Jennifer Greene. Applied Software Project Management. US: O'Reilly Media, 2005: 113-114
- [42] Karl E. Wieggers. Software Requirements, Second Edition. US: Microsoft Press, 2003: 23-25
- [43] Ralph R. Young. Effective Requirements Practices. Boston: Addison-Wesley, 2001: 120-123
- [44] 许晓冯. Web 应用系统的安全威胁及其防护. 信息化研究, 2009(12): 1-3
- [45] 张良银. 浅论 C/S 和 B/S 体系结构. 工程地质计算机应用, 2006(4): 20-23
- [46] 蔡宏伟, 金连甫, 陈平. 可扩展三层 B/S 体系结构研究和应用. 浙江理工大学学报(自然科学版), 2006, 23(2): 63-66
- [47] 徐兰芳, 彭冰, 吴永英. 数据库设计与实现. 上海: 上海交通大学出版社, 2006: 45-48